

Tema-1: Búsqueda de soluciones en IA. Heurísticos y Metaheurísticos.

El problema de la búsqueda de soluciones.

Satisfabilidad y Optimalidad. Explosión combinatoria.

Tipología de la Búsqueda:

Búsqueda Global (sistemática), Búsqueda Local.

Métodos Constructivos, Métodos de Mejora, Métodos Poblacionales.

Búsqueda Heurística.

Tipos. Algoritmos A, A*. Variantes.

Metaheurísticos.

Tipología Metaheurísticas. Exploración vs Explotación.

Elección de una metaheurística. No free lunch.

Bibliografía

- Monografía: Metaheurísticas. *Inteligencia Artificial, Vol 7, No 19 (journal.iberamia.org) (2003).*
 - Handbook of Metaheuristics, Springer 2ª ed. M.. Gendreau, J.Y.Potvin, 2010
 - Essentials of Metaheuristics. Sean Luke. 2013. (Online <http://cs.gmu.edu/~sean/book/metaheuristics/>)
 - Metaheuristics Network Website (<http://www.metaheuristics.org/>)
-

- Inteligencia Artificial. Técnicas, métodos y aplicaciones. Palma, Marín. McGraw Hill (2008)
 - Inteligencia Artificial. Un enfoque moderno. S. Russell, P. Norvig. Prentice Hall 3ª ed (2010).
 - Computational Intelligence. A.Engelbrecht. Wiley & Sons. 2ed. 2007 (<http://ci.cs.up.ac.za/>)
 - How to Solve It: Modern Heuristics. Z. Michalewicz, D. Fogel. 2ed. 2004 (Springer)
-

Diversos Artículos

Journal of Heuristics,

Int. Journal of Applied Metaheuristic Computing,

Int. Workshop on Hybrid Metaheuristics, etc.

Int. Journal of Metaheuristics,

Applied Intelligence, etc.

Bibliografía

- Monografía: Metaheurísticas. Inteligencia Artificial, Vol 7, No 19 (journal.iberamia.org) (2003).
 - Handbook of Metaheuristics, Springer 2ª ed. M.. Gendreau, J.Y.Potvin, 2010
 - Essentials of Metaheuristics. Sean Luke. Online. <http://cs.gmu.edu/~sean/book/metaheuristics/>
 - Metaheuristics Network web site (<http://www.metaheuristics.org/>)
-

- Inteligencia Artificial. Técnicas, métodos y aplicaciones. Palma, Marín. McGraw Hill (2008)
 - Inteligencia Artificial. Un enfoque moderno. S. Russell, P. Norvig. Prentice Hall 3ª ed (2010).
 - Computational Intelligence. A.Engelbrecht. Wiley & Sons. 2ed. 2007 (<http://ci.cs.up.ac.za/>)
 - How to Solve It: Modern Heuristics. Z. Michalewicz, D. Fogel. 2ed. 2004 (Springer)
-

Diversos Artículos

- Journal of Heuristics, Int.Journal of Metaheuristics, Int.Journal of Applied Metaheuristic Computing, etc.
- Int. Workshop on Hybrid Metaheuristics, etc.
- Experimental Evaluation of Heuristic Optimization Algorithms: A Tutorial
Ronald L. Rardin and Reha Uzsoy, Journal of heuristics, 7(3), 2001.

El problema de la búsqueda de soluciones: Obtención de Soluciones Mediante Búsqueda.

La generación de “buenas” soluciones es un método típico para la resolución de problemas en IA:

Planificación (& robots), Diseño y Configuración, Respuestas a preguntas, Sistemas de Decisión, Juegos, Aprendizaje Inductivo (de ejemplos), Generación de rutas, N-recubrimiento, Asignación de recursos, etc.

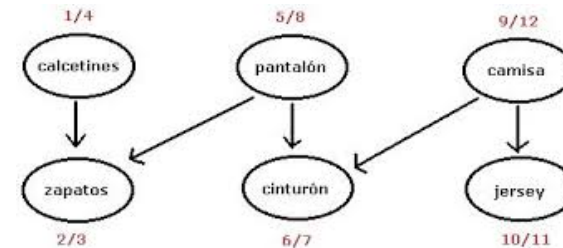
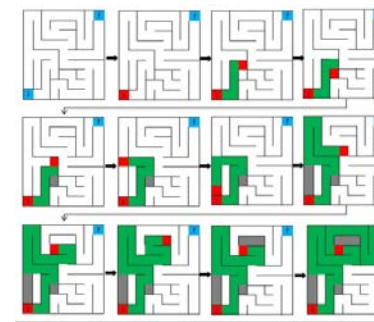
La solución debe ser **obtenida** (*generada, mejorada*), mediante un proceso de búsqueda en un **amplio espacio de estados/soluciones**.

- **Métodos matemáticos:** capaces de solucionar complejos problemas (de optimización) en tiempos computacionales bajos.

Pero hay problemas para los que **no existe o no es viable un método exacto**.

- Los problemas de optimización son particularmente **difíciles de resolver**.

Satisfabilidad: NP-completa, Optimalidad: **NP-dura**.



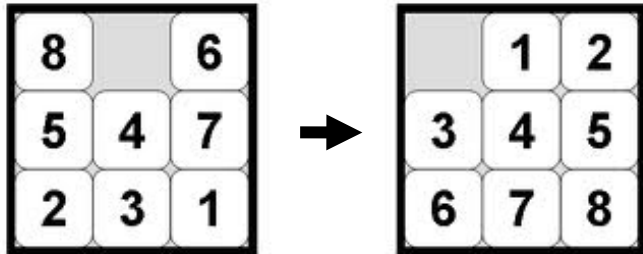
A menudo **no se puede** garantizar encontrar la solución óptima en tiempo computacional razonable.

⇒ **Alternativa?**: Obtener una **buena solución** (**optimalidad**), pero no necesariamente la óptima.

Problemas de optimización sujetos a restricciones: Obtención de soluciones que:

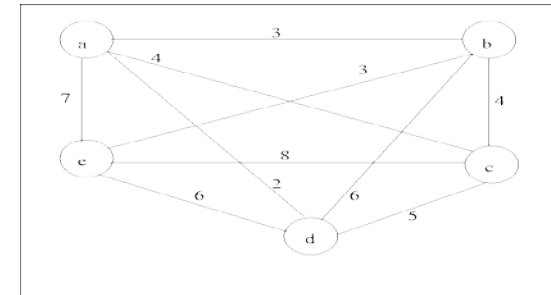
- Cumplan los criterios de factibilidad (satisfagan las restricciones/condiciones del problema)
- Maximicen una determinada función objetivo (optimalidad).

8-puzzle



Difícil encontrar una solución (problema de factibilidad), pero también existen soluciones más óptimas que otras (problema de optimalidad).

Problema del Viajante



Es trivial encontrar soluciones factibles, el problema es encontrar soluciones optimizadas (problema de optimalidad).

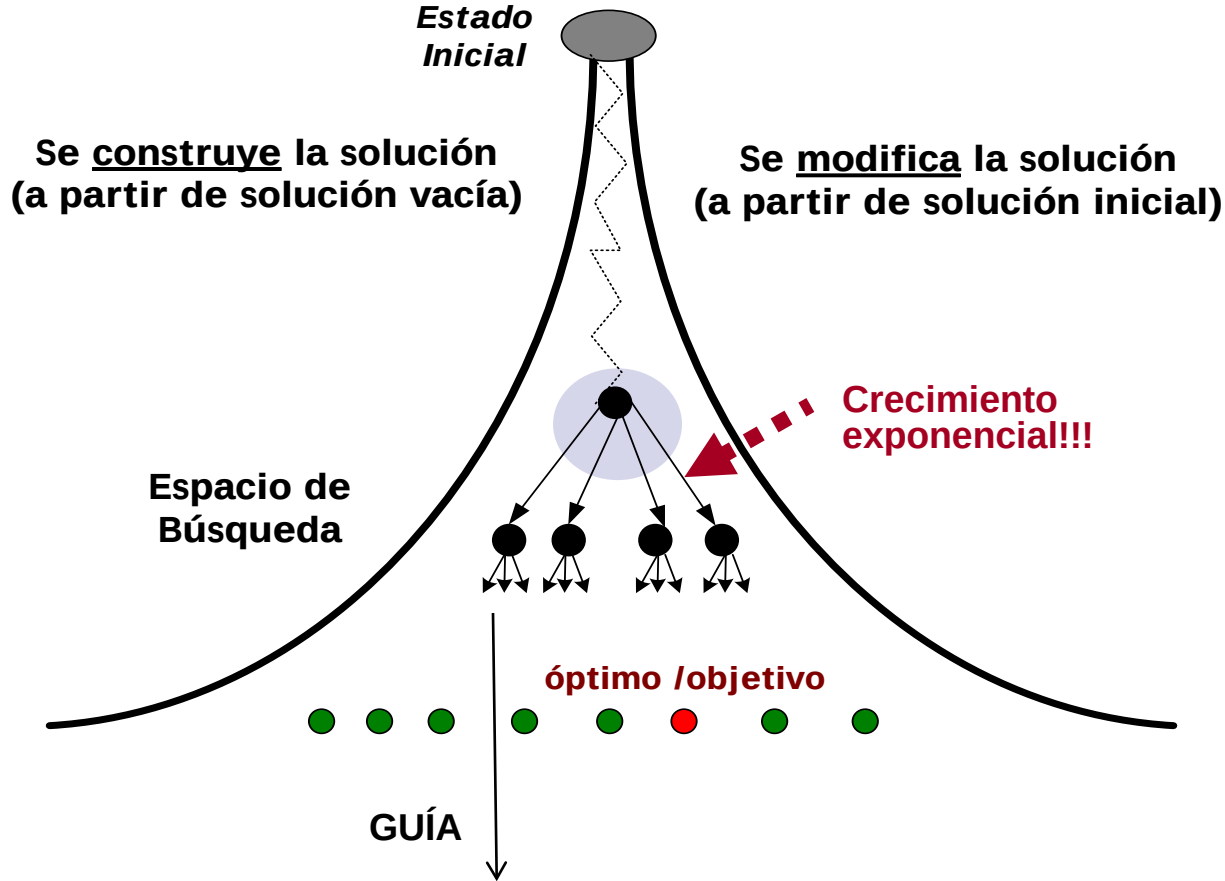
En general, **no es viable** un método exacto que garantice encontrar la solución óptima a **problemas realistas** en tiempo computacional razonable.

- En general, basta con una solución **razonablemente buena** (*factible, pero no la óptima*)
- Aplicación de métodos aproximados (como alternativa a los métodos exactos)

Métodos Heurísticos, Metaheurísticos

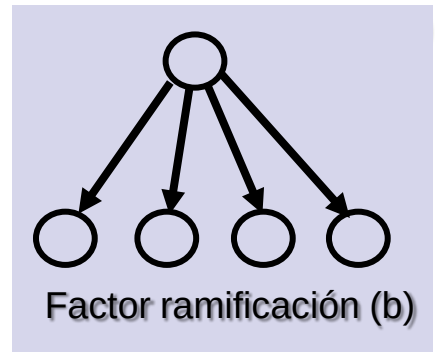
Procedimientos eficientes para encontrar (buenas) soluciones (aunque no sean óptimas):

Búsqueda de Soluciones $\Rightarrow$ Explosión Combinatoria (*alternativas decisión, MIP*)



*Planificación (robots),
Diseño y Configuración,
Respuestas a preguntas,
Sistemas de Decisión,
Juegos,
Aprendizaje Inductivo,
Generación de rutas,
Recubrimiento y
Empaquetamiento,
Asignación de recursos, etc.*

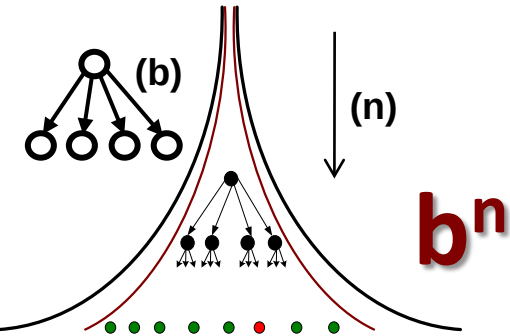
Alternativas en la Búsqueda
 $\Rightarrow$ **Crecimiento Exponencial**



Búsqueda Heurística / Metaheurística:

La solución es obtenida (*generada, mejorada*), mediante un proceso de búsqueda guiado en un amplio espacio de estados/soluciones.

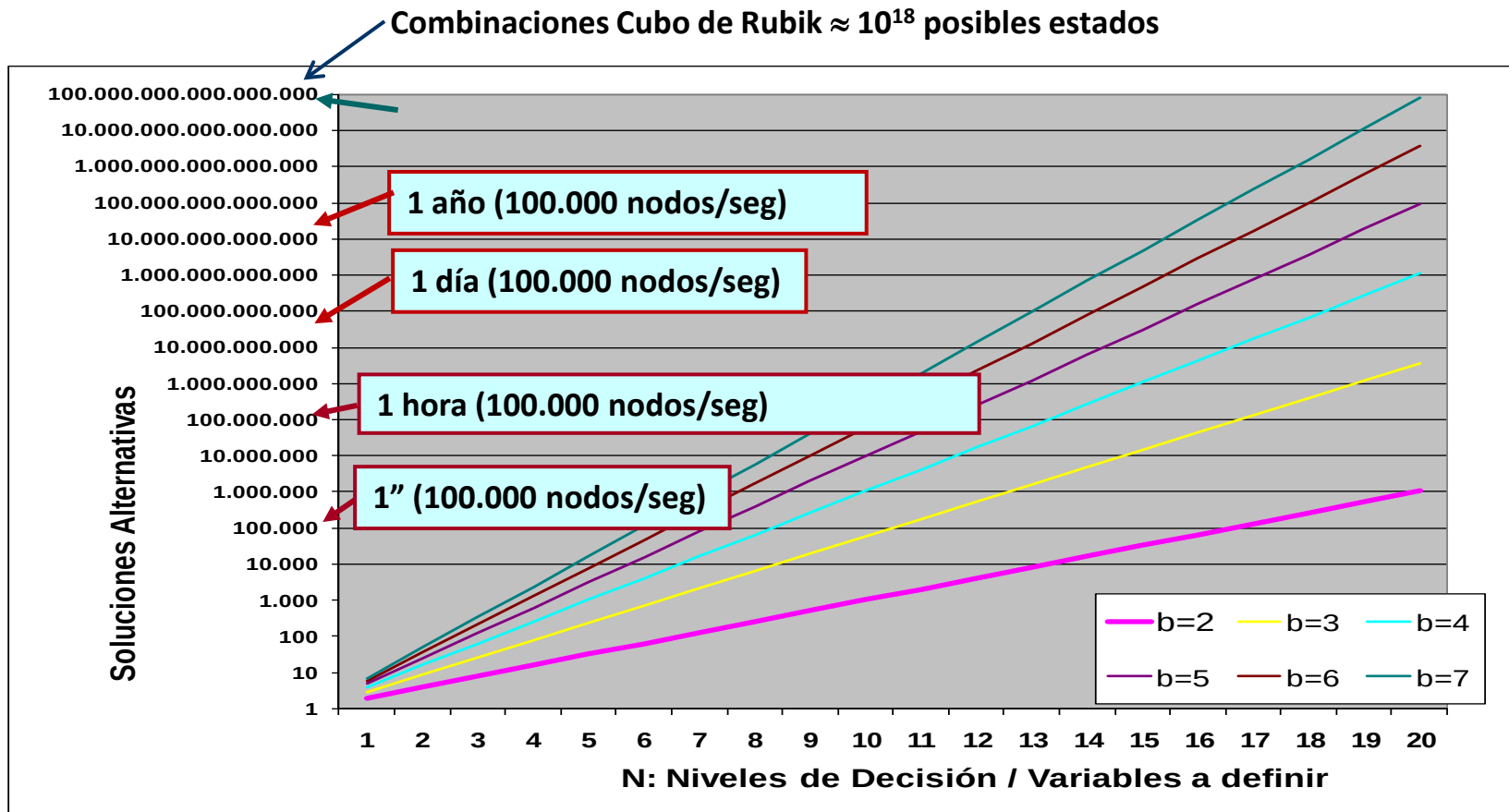
BÚSQUEDA $\Rightarrow$ explosión combinatoria.



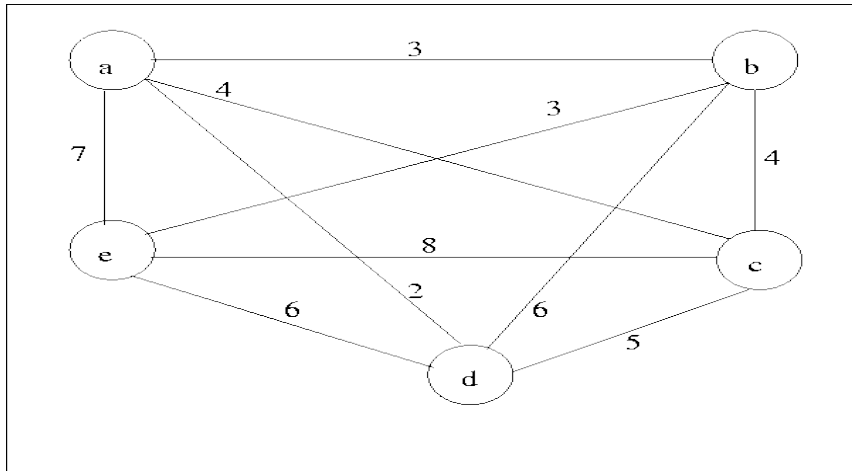
La explosión combinatoria depende del factor de ramificación **(b)** y niveles de decisión **(n)**.

En dominios finitos (optimización combinatoria), el conjunto de soluciones es finito y numerable (pero muy grande).

Una fina hoja de papel (0,1 mm), doblada 50 veces, alcanza un espesor de....



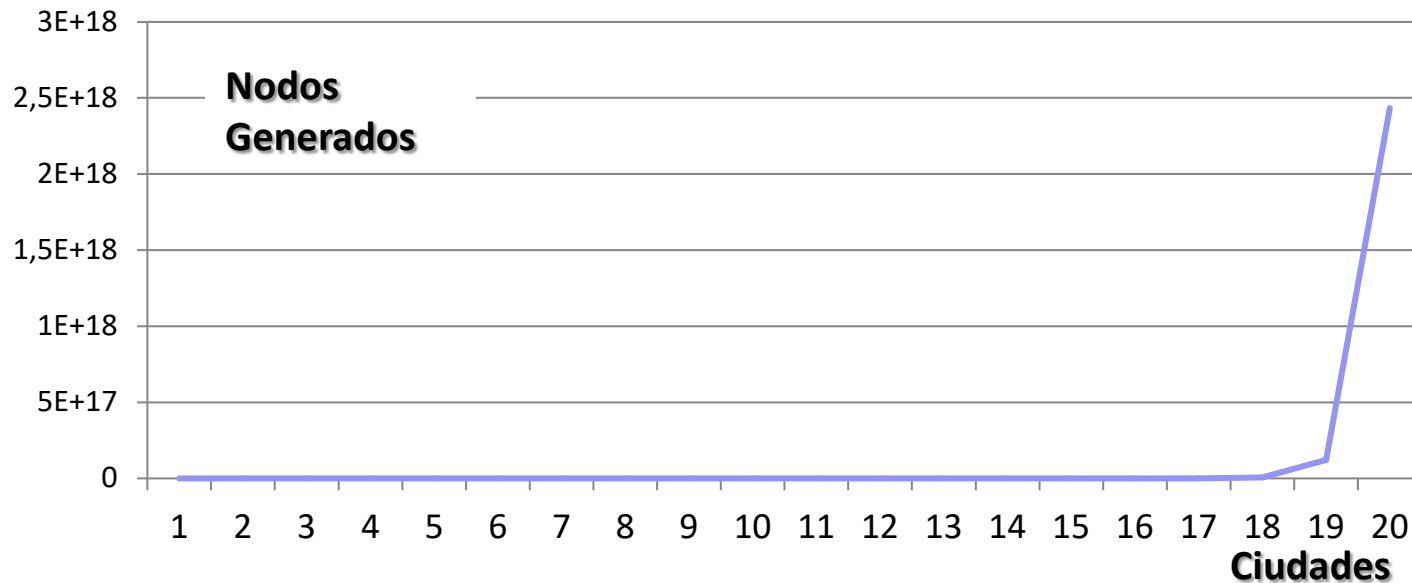
Un típico ejemplo (Problema del Viajante):



Coste: $\text{Fact}(n)$

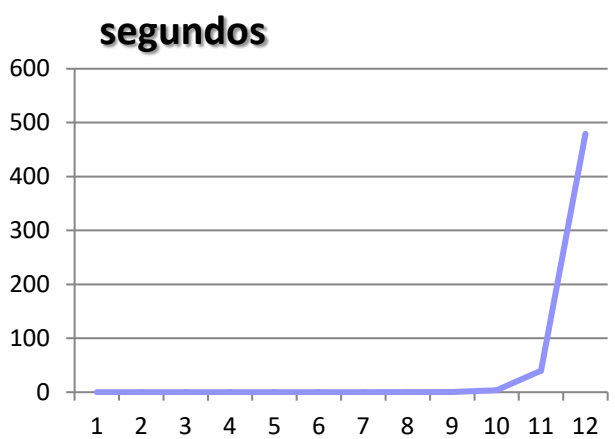
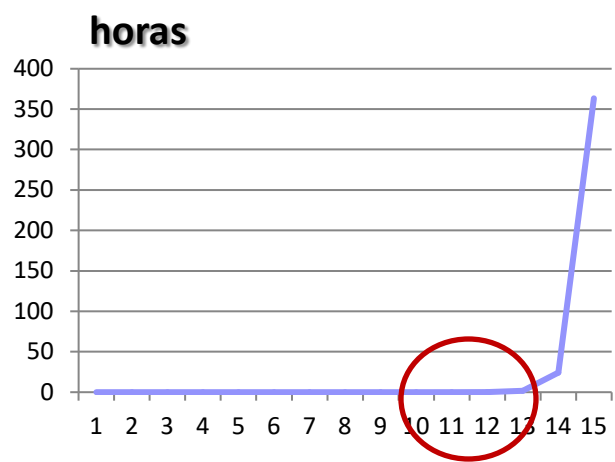
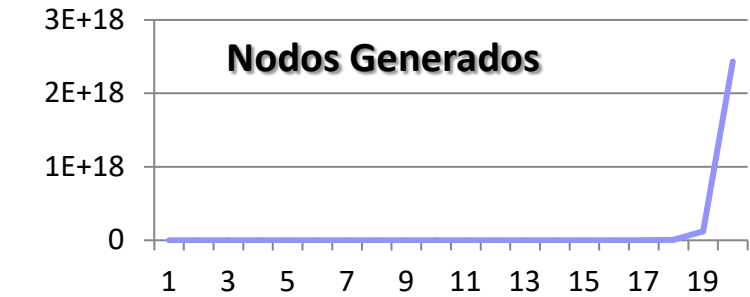
$\text{Fact}(5) = 120$ soluciones

Pero, y con 20 ciudades?



Es mucho o es poco?

Con 20 ciudades, generando 1.000.000 de circuitos/seg



Es un problema de Crecimiento Exponencial:
Alternativa: Métodos aproximados (como alternativa a los métodos exactos)

Estrategias de Búsqueda

(Global / Local)

Búsqueda Sistemática (Global):

busca en **todas** las alternativas posibles.
(tentativa, backtracking, algoritmos A, ...)

Búsqueda Local :

busca solo en el **entorno local actual**.
(irrevocable, escalada, ...)

Métodos (procesos)

(Búsqueda en un Espacio
de
Estados / Soluciones)

Métodos Constructivos: construyen una solución paso a paso

Búsqueda heurística iterativa elementos de la solución.
Espacio de búsqueda $\Rightarrow$ espacio de soluciones parciales

Métodos de Mejora: mejora iterativa de soluciones.

Búsqueda heurística iterativa nueva soluciones cercanas.
Espacio de búsqueda $\Rightarrow$ espacio de soluciones.

Métodos Evolutivos/Enjambre: provienen de soluciones previas

Generación de nuevas soluciones utilizando información implícita en la sociedad/población de soluciones previas.

Estrategias de Búsqueda: Criterio para determinar el siguiente estado en la búsqueda

Estrategias de Búsqueda Global (*global search*):

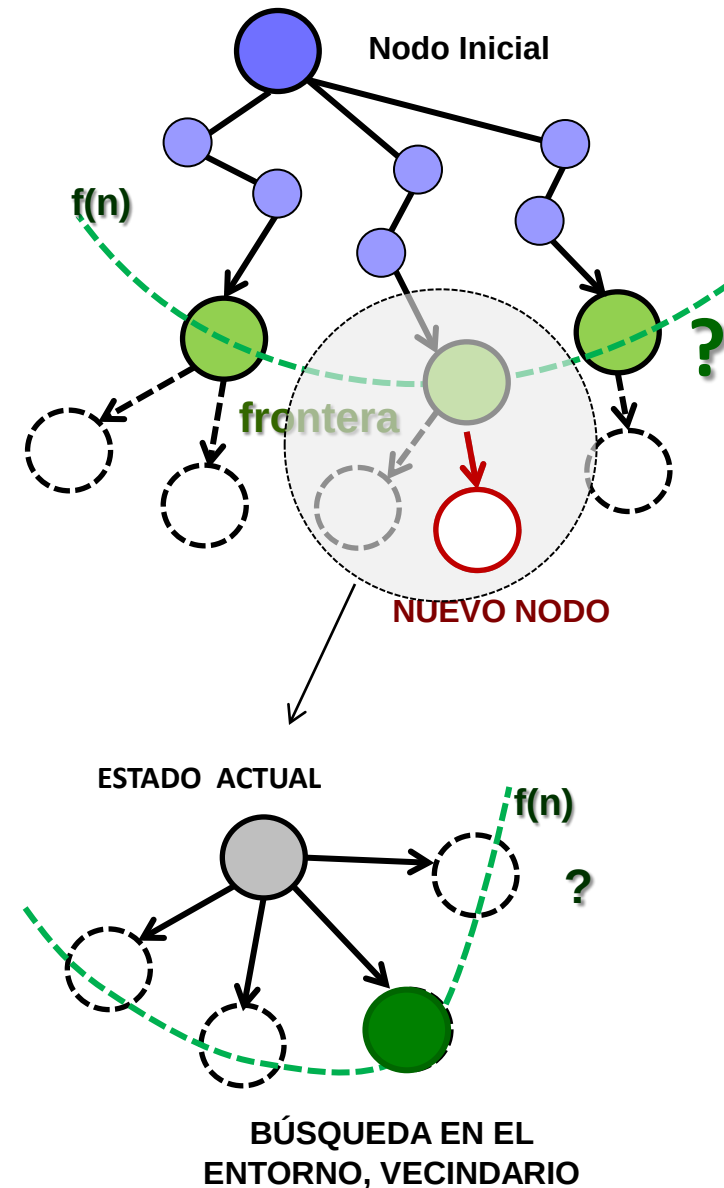
Búsqueda sistemática del espacio de búsqueda: por **cualquier nodo (frontera) del espacio de búsqueda**.

- ⇒ **Alto coste en memoria** (*explosión combinatoria*).
- Puede ser completa y admisible (garantizar solución óptima)
- Típicamente se usan para generar soluciones.

Estrategias de Búsqueda Local (*local search*):

En cada iteración, se busca solo en el **entorno del estado actual**.

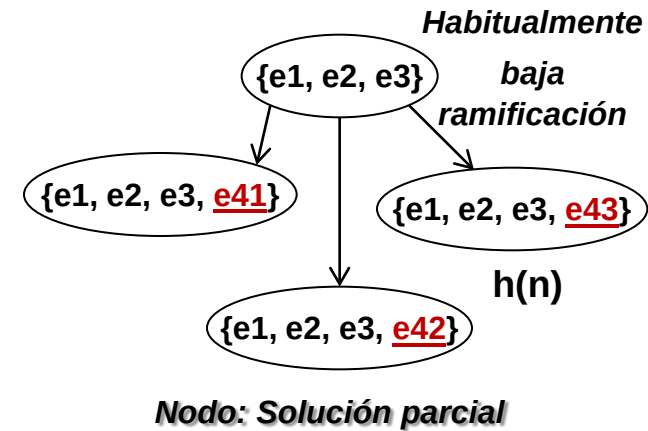
- Retienen solo el estado actual ($\approx$ esquema irrevocable).
⇒ Eficientes en **memoria**..
- Pueden utilizarse para **generar** una solución o **mejorar** una solución previa.
- Requieren definición de **vecindario** de una solución.
- **Inconvenientes:** Pueden quedar atrapadas en soluciones que no admiten mejoras en su entorno (óptimos locales, valles, mesetas), problemas de incompletitud, ciclos, etc.



Métodos de Búsqueda: Método para la obtención de una buena solución.

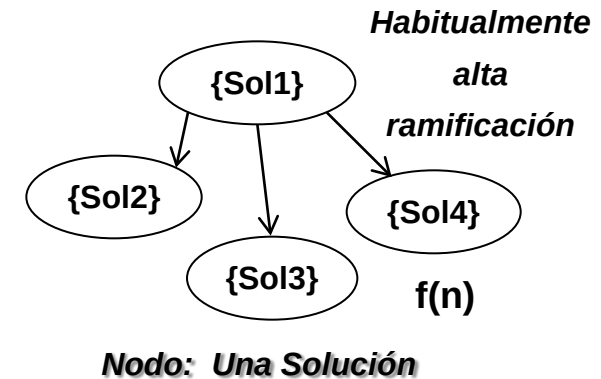
Métodos Constructivos

- **Construyen** iterativamente una solución al problema (*≈ Búsqueda en Sistemas Basados en Reglas*)
- Cada paso añaden (**heurísticamente**) un elemento a la solución parcial.
- **No requieren solución inicial:** de aplicación en problemas en donde lo difícil es obtener una solución: **Problemas de factibilidad**.
- Puede aplicarse con una búsqueda local o búsqueda global.



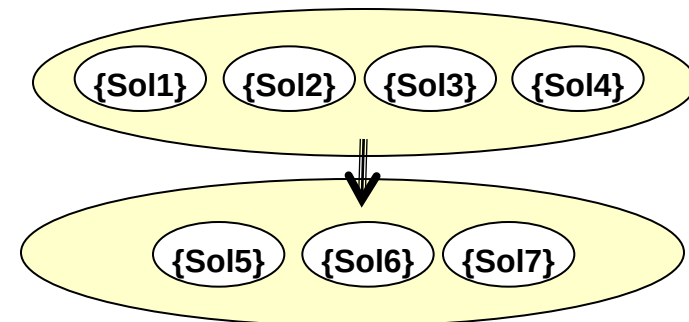
Métodos de Mejora (*habitualmente llamados Búsqueda Local*)

- Requieren solución inicial (constructiva / aleatoria).
- Cada iteración modifica solución actual (eligiendo **heurísticamente** en su entorno) intentando **mejorarla**.
- Ramificación muy alta $\Rightarrow$ búsqueda local.
- Ventaja *any-time*.

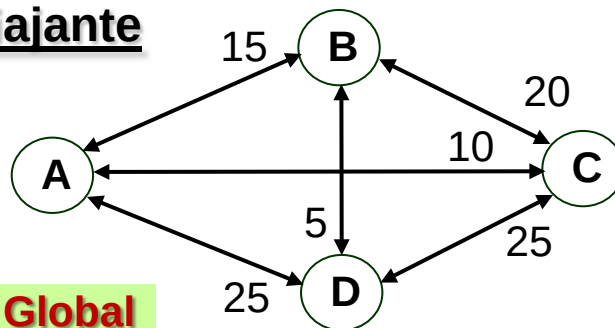


Métodos Evolutivos / Enjambre

- Parten de un conjunto (**población/enjambre**) de soluciones, que se seleccionan, recombinan, modifican o colaboran para obtener un nuevo conjunto de soluciones.
- Ventaja *any-time*.



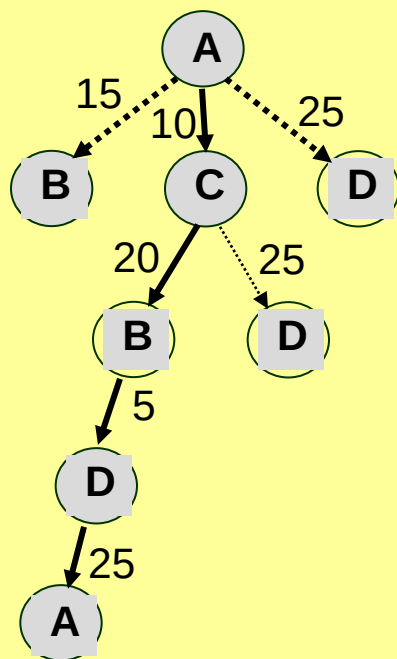
Ejemplo: Problema del Viajante



3	8	4
1	?	5
7	2	6

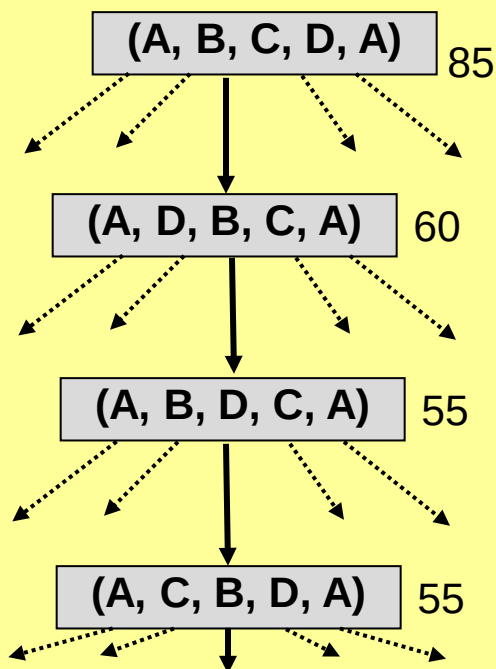
Búsqueda Local / Búsqueda Global

Método Constructivo



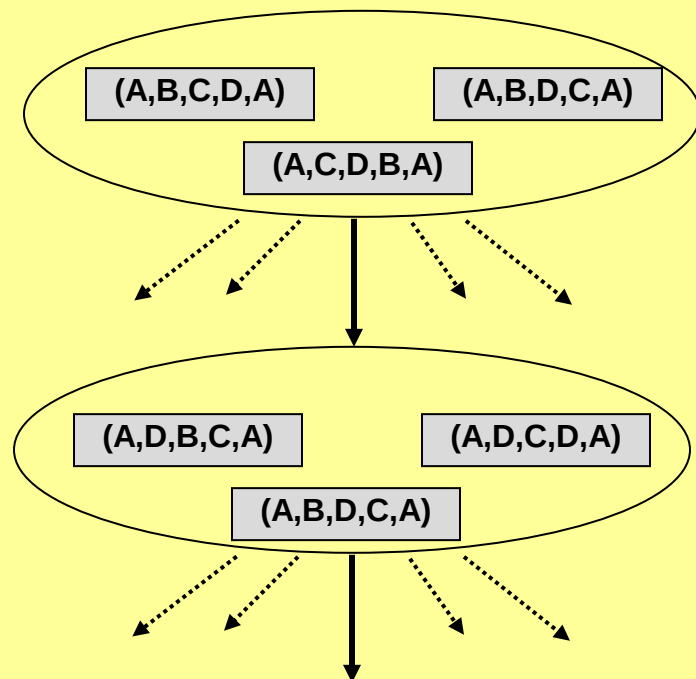
Sucesores:
Nuevos elementos
Típicamente pocos

Método de Mejora



Sucesores:
Soluciones alternativas
Típicamente muchos

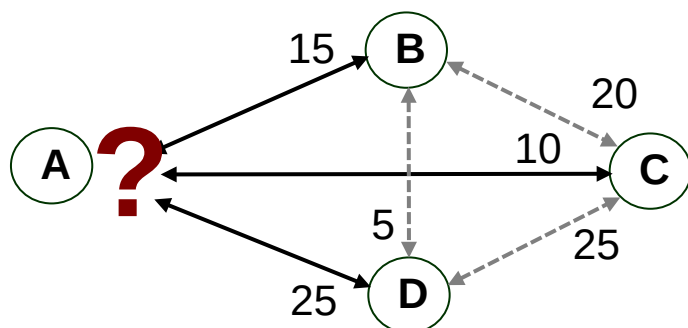
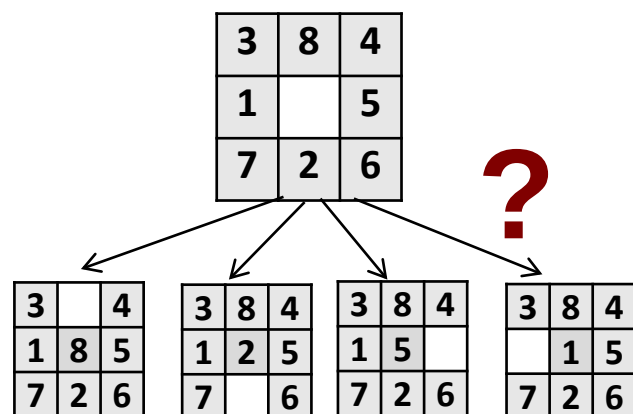
Método Evolutivo



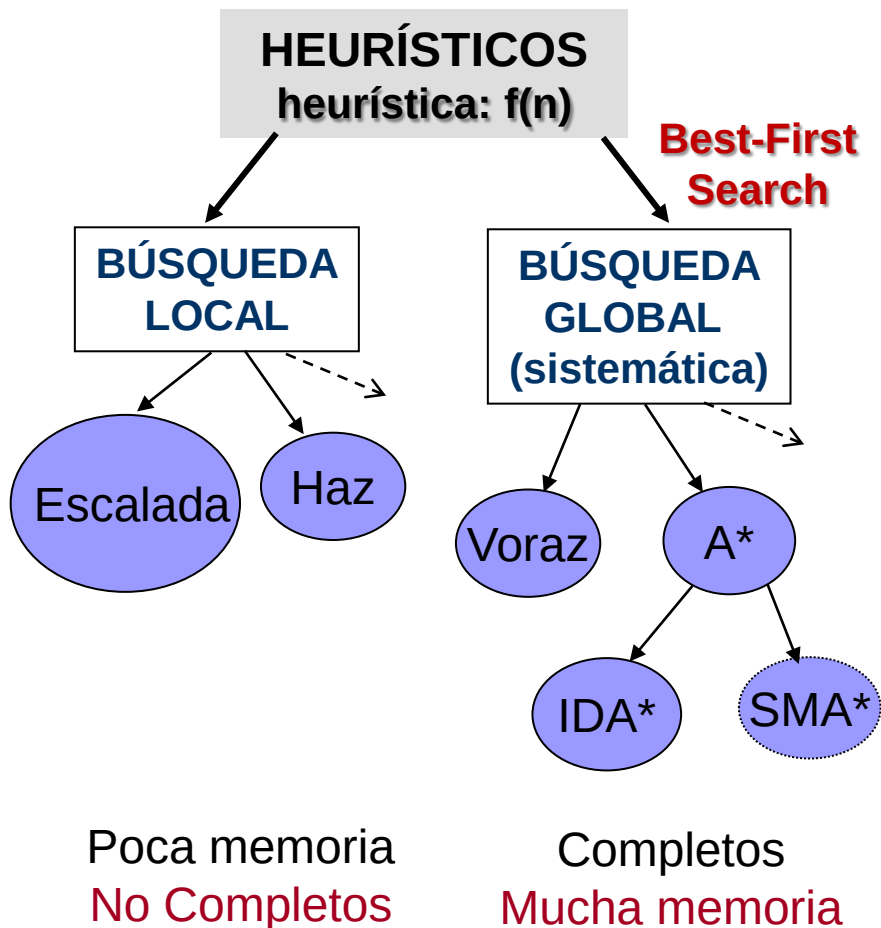
Heurística: Guía en la búsqueda de soluciones

Criterios aplicados a la resolución de problemas complejos, esperando obtengan una buena solución (no necesariamente óptima) de un modo sencillo y rápido.

Métodos aproximados, contrapuestos a los exactos. No son resultado de un riguroso análisis formal, sino de **conocimiento intuitivo**, experimental, práctico, de experto, etc. sobre el problema.



- Pueden ser **generales** o **dependientes** del problema (más eficientes, menos generalizables).
- ✓ **Relajación** (relajación de condiciones/restricciones del problema),
- ✓ **Inductivas** (generalización de soluciones a versiones sencillas del problema),
- ✓ **Descomposición** (descomposición en subproblemas más sencillos de resolver),
- ✓ **Reducción** (identificar propiedades que se cumplen mayoritariamente en las buenas soluciones e introducirlas como restricciones del problema).
- ✓ **Abstracción** (abstracción de estados distintos del problema en un único estado).
- ✓ **Landmarks** (identificación de estados intermedios que deben ser alcanzados)



Búsqueda Heurística

Aplica una función $f(n)$, que mide la bondad del nodo (estado), y tiene una componente heurística $h(n)$:

$h(n)$ = estimación del coste al objetivo.

$$\forall n, h(n) \geq 0, h(\text{meta}) = 0$$

Según las Estrategias:

➤ Búsqueda Local (Escalada, Haz):

- Requieren muy poca memoria;
- Pero, **problemas** de valles, ciclos, etc.... y **no garantiza la óptima**.

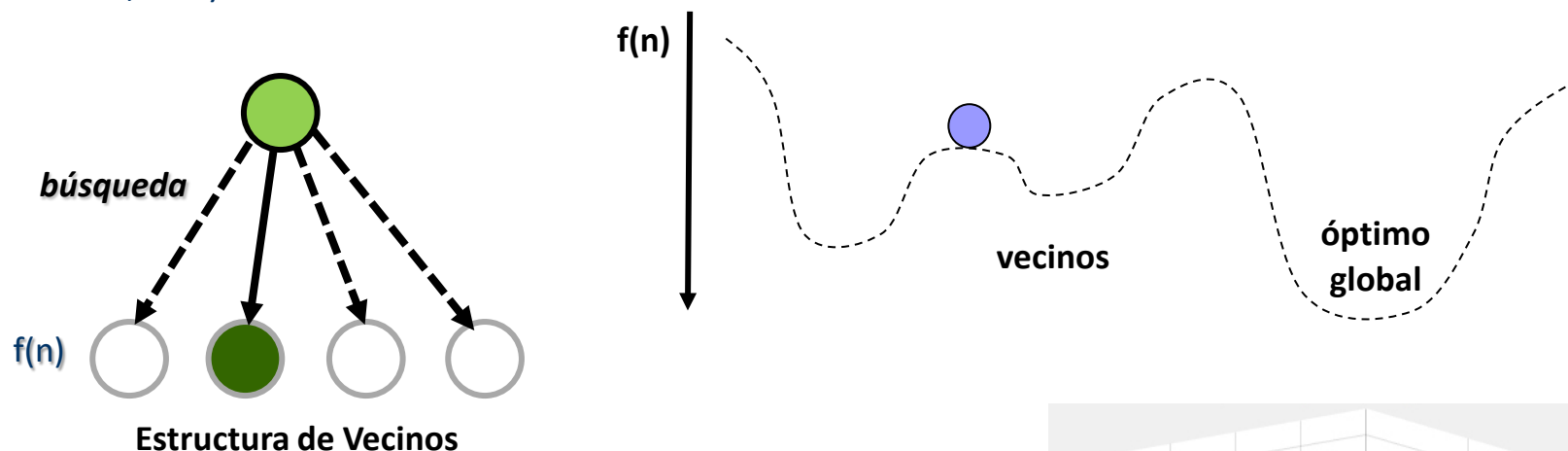
➤ Búsqueda Global (en grafo/arbol)

- Primero el mejor (*Best First Search*): expande el mejor nodo frontera.
- Permite soluciones óptimas, pero suele requerir mucha memoria.

Búsqueda Heurística en IA: Local, No sistemática.

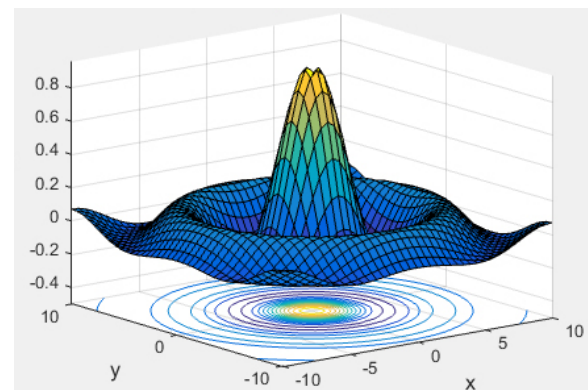
Función de Escalada (*Hill-climbing*) , Descenso por Gradiente, Voraz local, etc.

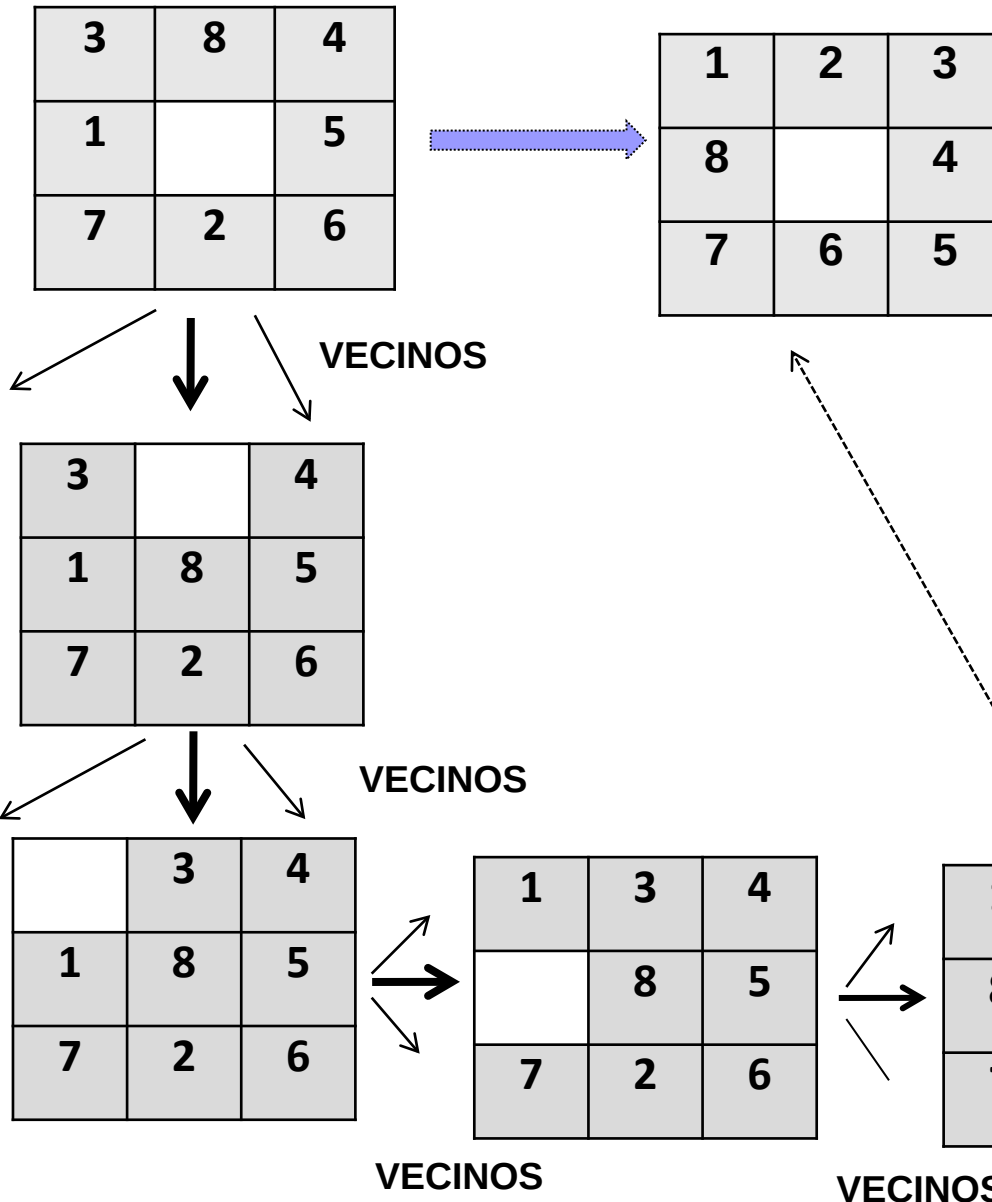
- Selección de un sucesor (**vecino**) que mejore $f(n)$ respecto al padre: escalada por máximos locales.
- Local: No mantiene un árbol de búsqueda, sino **sólo el estado actual** (y sucesores inmediatos).
- Realiza unos bucles de búsqueda dirigido hacia el **crecimiento de $f(n)$** (colina arriba, máximo gradiente). El proceso **acaba** cuando no hay mejora posible en el conjunto de soluciones vecinas (valles, llanuras, ciclos, etc.).



Ventaja: Memoria limitada ($\cong$ constante), no necesita mantener caminos alternativos.

Inconvenientes: óptimos locales (no garantiza óptimo global, valles, mesetas, ciclos, etc...)

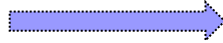




Hill climbing algorithm

1. $n = \text{initial node}$.
2. While $n \neq \text{goal}$.
 If $\text{Sucessors}(n) = \emptyset$, finish with failure
 Select $n' \in \text{Sucessors}(n)$ to minimize $f(n')$.
 If $f(n) < f(n')$, finish with failure,
 else $n = n'$

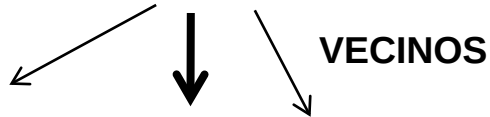
3	8	4
1		5
7	2	6



1	2	3
8		4
7	6	5

Hill climbing algorithm

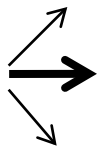
1. $n = \text{initial node}$.
2. While $n \neq \text{goal}$.
 If $\text{Sucessors}(n) = \emptyset$, finish with failure
 Select $n' \in \text{Sucessors}(n)$ to minimize $f(n')$.
 If $f(n) < f(n')$, finish with failure,
 else $n = n'$



3		4
1	8	5
7	2	6

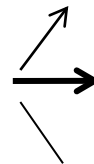
VECINOS

	3	4
1	8	5
7	2	6



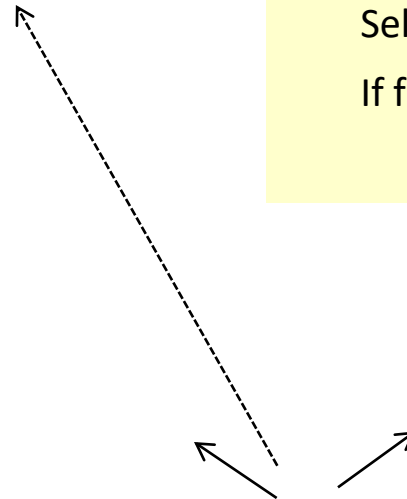
VECINOS

1	3	4
	8	5
7	2	6

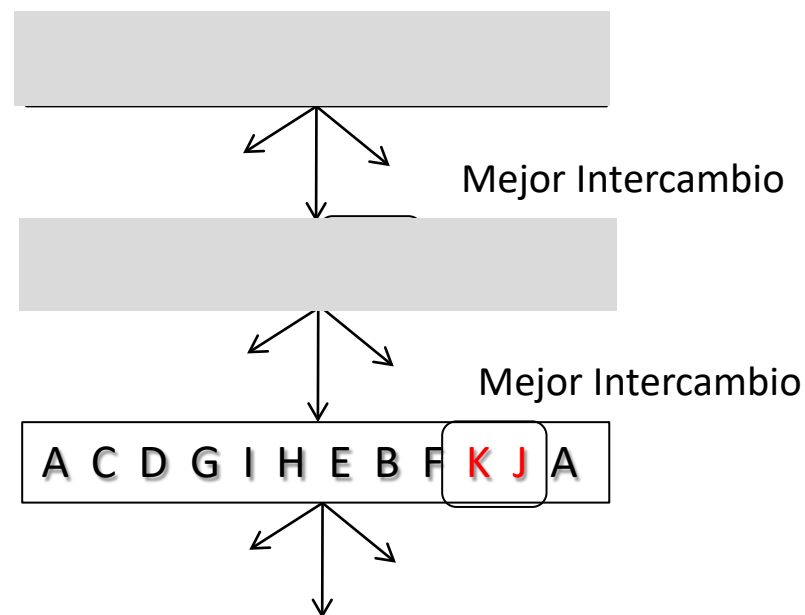
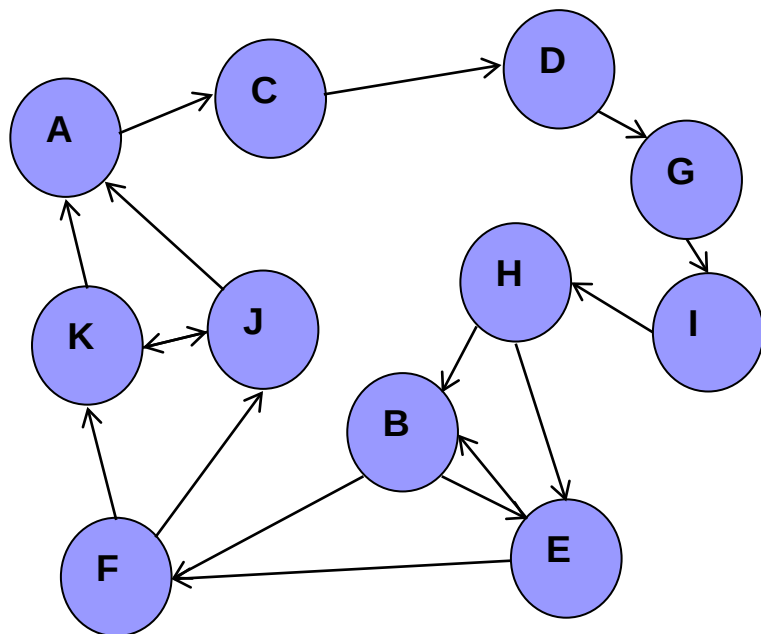


VECINOS

1	3	4
8		5
7	2	6



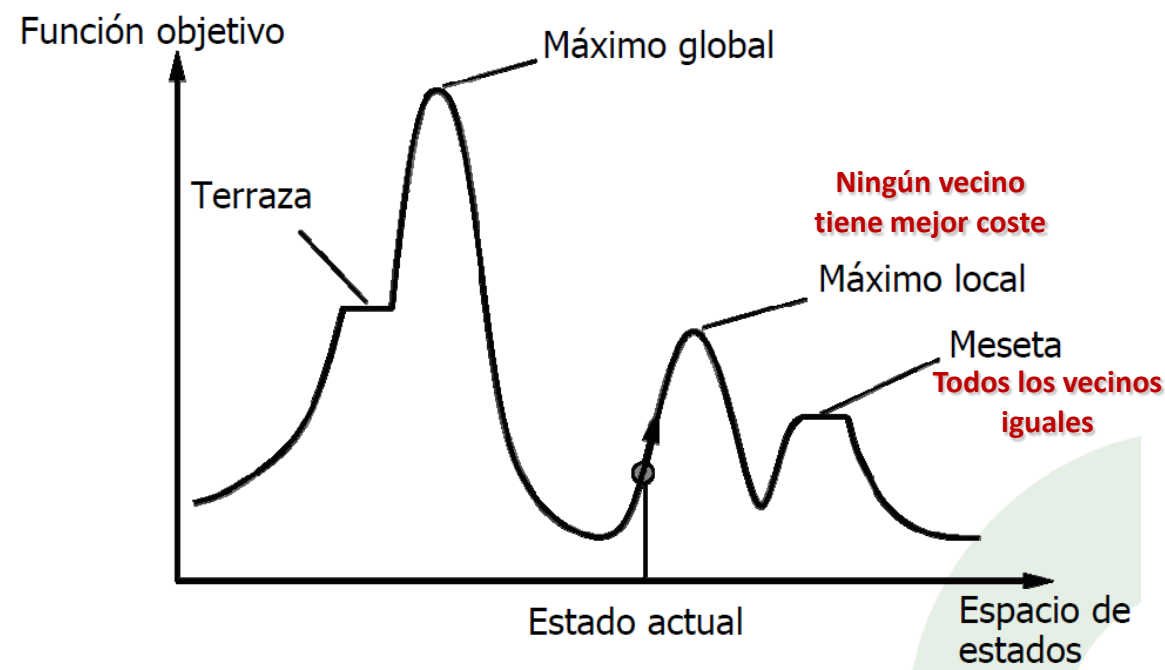
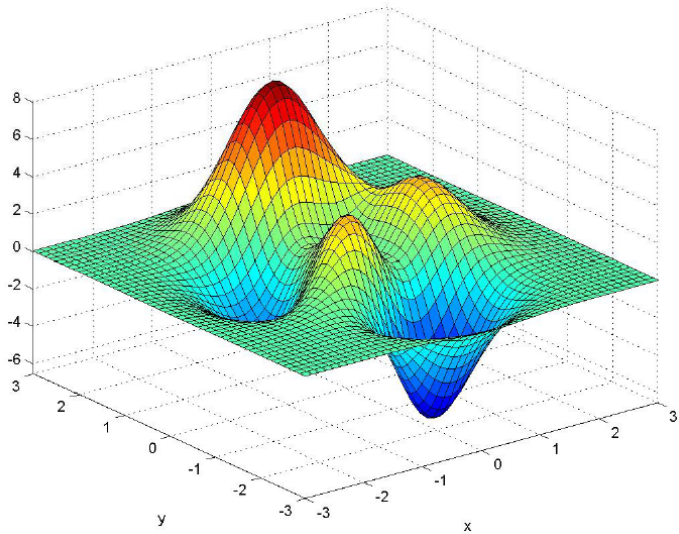
Ejemplo: Escalada (Vecinos: Intercambio de pareja)



**Ningún intercambio mejora:
Parada Máximo Local**

Inconvenientes:

- No hay memoria en la búsqueda, no hay visión global.
- La guía $f(s)$ es fija.
- Búsqueda puede quedar atascada (pararse) en **óptimo local** (todos los vecinos peores), **llanuras** (todos los vecinos iguales), **bucles** (debido a no almacenar en memoria).
- Se obtienen soluciones localmente óptimas, pero pueden estar muy lejos del óptimo global.



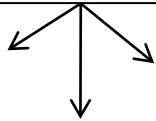
Después de escalar una gran colina, uno se encuentra sólo con que hay muchas más colinas que escalar. Nelson Mandela



Posibles Soluciones: (escape de un óptimo local, en cuyo entorno no existen mejores valores que el actual)

- Salto a otra zona de búsqueda (previamente guardada) $\approx$ (Búsqueda en Haz, Beam Search)
- Avanzo al siguiente nivel, o más niveles (peores!!)
- Re-arranque $\approx$ Random Restarting

A C D G I H E B F **K J A**

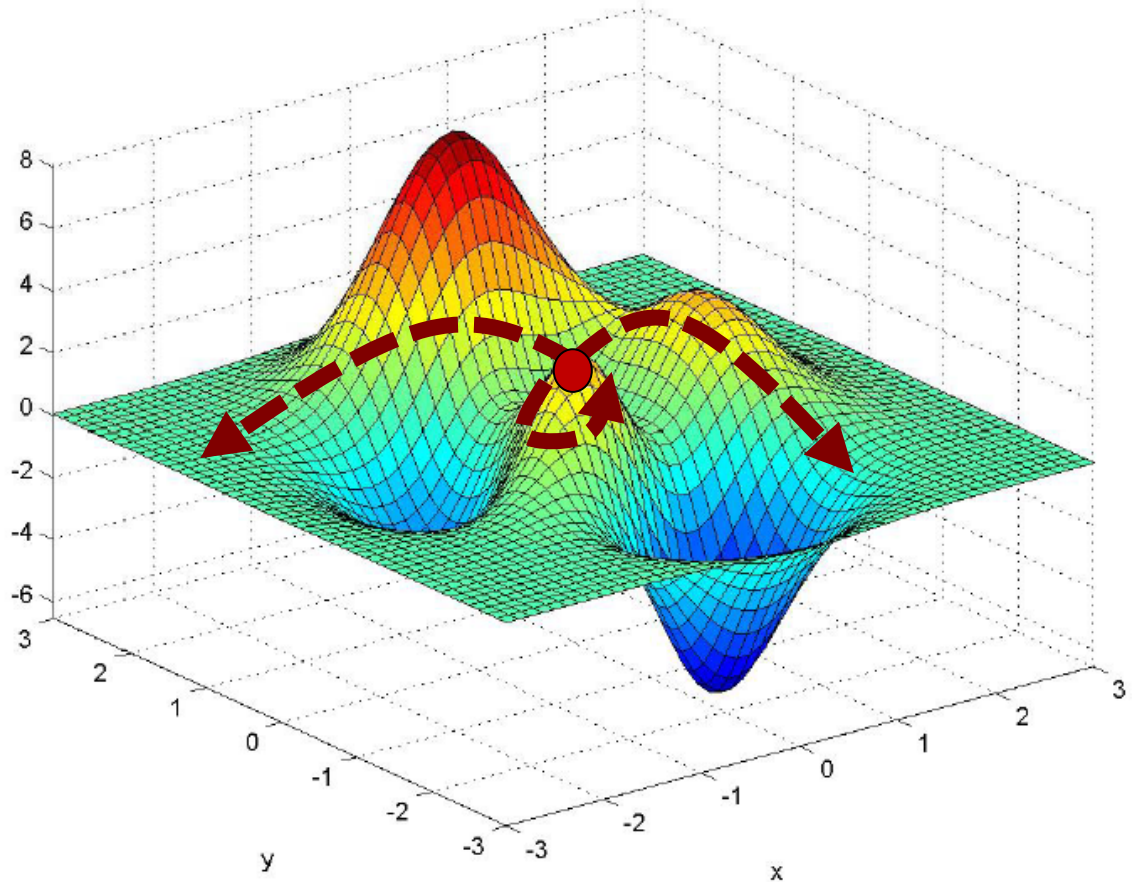


Ningún intercambio mejora

Metaheurísticas

Combinación

Explotación / Exploración



Búsqueda Heurística en IA: Global, Sistemática

Mantienen abierta la frontera de búsqueda: árbol/grafó de búsqueda

Búsqueda Voraz, A y A* son ‘Búsquedas Primero el Mejor’, Sistemáticas

Búsqueda Voraz (Greedy).

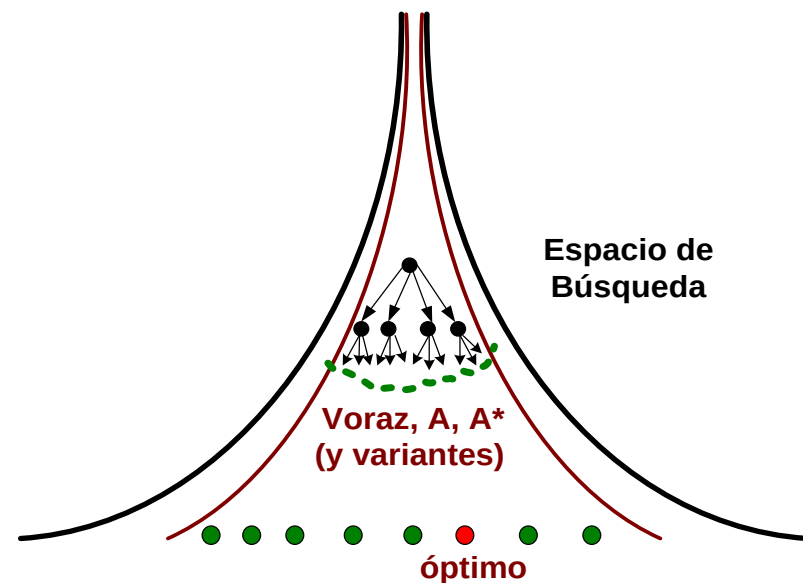
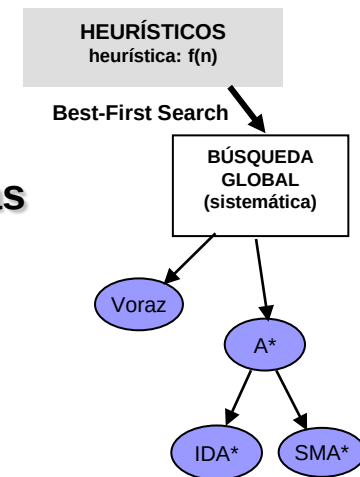
- Caso típico de “búsqueda el primero mejor”, donde no importa $g(n)$:
$$f(n) = h(n)$$
- No siempre encuentran solución o acaban, no completos, no admisibles

Algoritmo A.

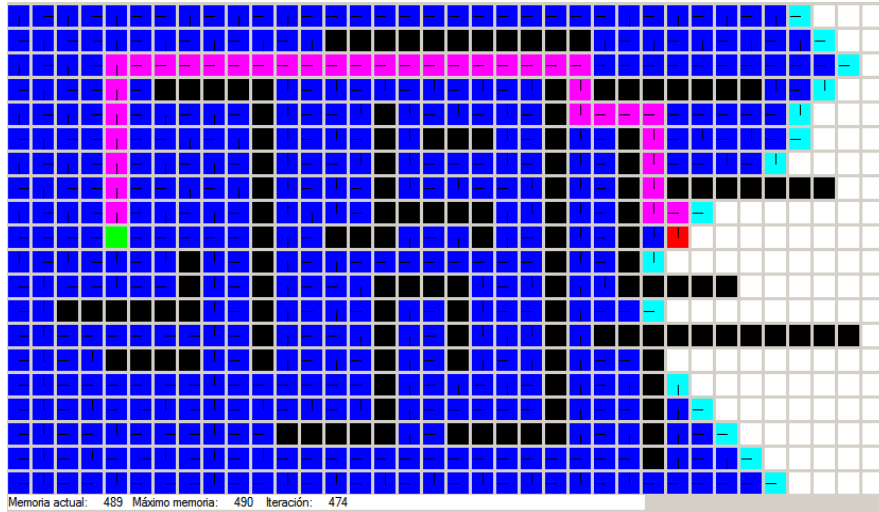
- Combinación de **búsqueda voraz** (reduce coste de búsqueda, pero no óptima ni completa) y **coste uniforme** (completa y óptima, pero ineficiente).

$$f(n) = g(n) + h(n)$$

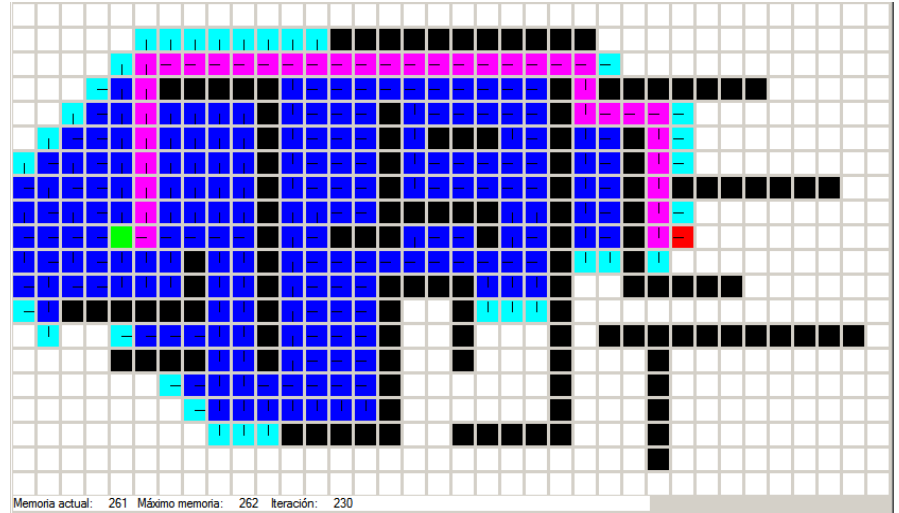
- **Algoritmo A***: $\forall n, h(n) \leq h^*(n)$
- **Completa y admisible**. ($h(n)=0 \cong$ Dijkstra, coste uniforme)
- **Pero requiere mucha memoria** (un algoritmo que expanda menos nodos que A* no tendrá admisibilidad).



Búsqueda A (h=0)



Búsqueda A (h(n): manhattan)



Además del tiempo computacional,
hay un problema de memoria

Variantes Algoritmo A:

Weighted A*

- $f(n) = g(n) + \alpha h(n)$, donde $\alpha > 1$
- Es una búsqueda completa, pero no admisible
- Se garantiza encontrar una solución menor que ' α ' veces el coste de la óptima.

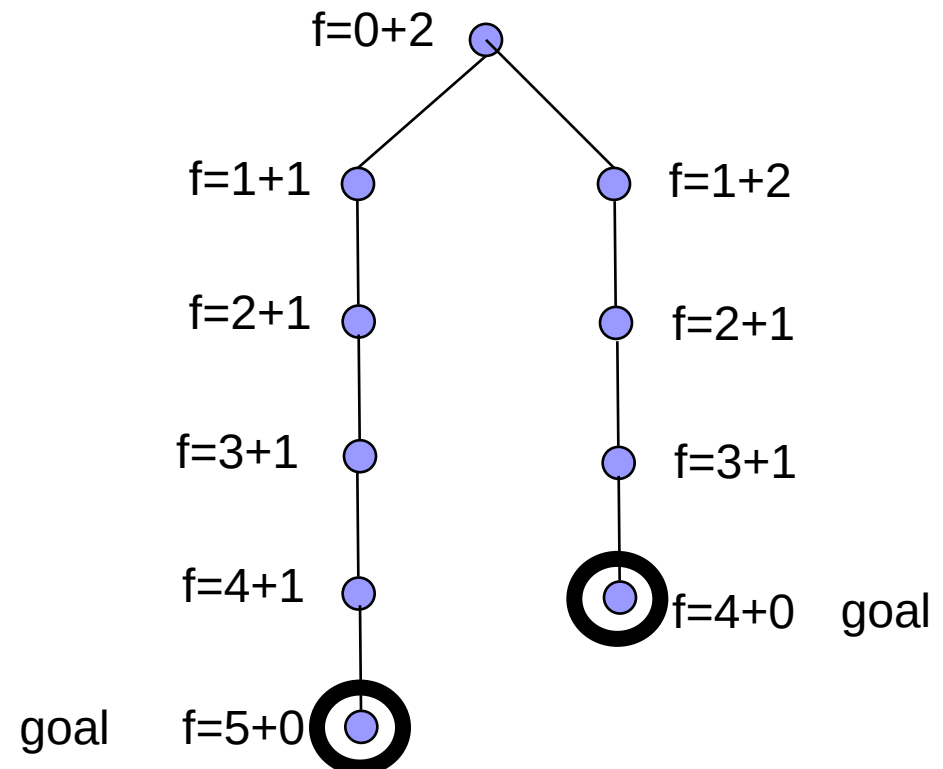
Variantes Algoritmo A para ahorrar memoria.....

Iterative-Deepening A* (IDA*)

- Adapta la Búsqueda en Profundidad Iterativa (BPI) a A*.
- El **criterio de corte** no es el nivel (profundidad), sino el valor de $f(n) = g(n) + h(n)$
- Cada interacción se reinicia desde la raíz, con un valor de corte: **coste (g+h) más pequeño de los nodos que fueron cortados en la iteración anterior.**

En cada iteración, cuando el valor $f(n)$ de un nodo supera el valor de corte, se aplica backtracking.

- Si $h(n) \leq h^*(n)$ IDA* es admisible
- Útil para problemas de costes $g(n)$ unitarios
- IDA* es completo y óptimo y necesita **menos memoria** que A* (es un iterativo en profundidad), aunque puede generar más nodos en total.
- **Complejidad Temporal:** $O(b^{2d})$
- **Complejidad Espacial:** $O(b \cdot d)$



Variantes Algoritmo A para ahorrar memoria.....

Simplified Memory-Bounded A* (SMA*)

Búsqueda con memoria acotada: determina un *máximo número de nodos a almacenar* (Nmax).

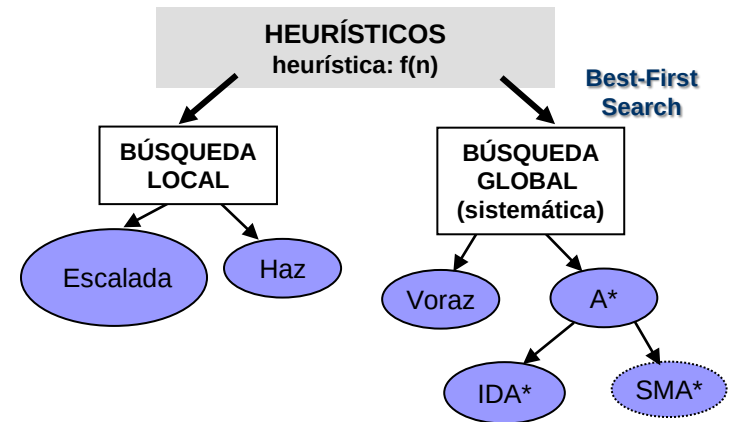
Modificación del algoritmo A*, de forma que cuando no puede almacenar más nodos, se elimina de la frontera del grafo (*lista open*) el nodo que tiene el mayor valor de $f(n)$: *'nodo olvidado'*

- En cada nodo se recuerda el hijo *'olvidado'* con mejor $f(n)$.
- El algoritmo puede *'recuperar'* los nodos olvidados si resultan más prometedores conforme avance la búsqueda (el resto de nodos tiene mayor $f(n)$).

Características

- Es capaz de adaptarse a la memoria disponible.
- *Es completo, si la memoria disponible es suficiente para almacenar la senda más corta.*
- Mejor para costes $g(n)$ no unitarios.
- *Es admisible, si la memoria disponible es suficiente para un camino óptimo.*
- En otro caso, devuelve la mejor solución que puede alcanzar con la memoria disponible.

En resumen:



Métodos exactos (no heurísticos)	Inviabiles en problemas complejos
Búsqueda heurística local	No Admisibles , Poca memoria Problemas de ciclos, etc.
Búsqueda Heurística Global: Algoritmo A, A	Búsqueda Global, sistemática Admisibles y Completos. Alto coste memoria / tiempo
Incrementar potencia heurística $h(n) > h^*(n)$ o Variantes Algoritmo A*	Pierden Admisibilidad? Bajan / Acotan coste memoria Siguen requiriendo mucha memoria
Metaheurísticos: Diseño y estrategias heurísticas	Soluciones Optimizadas Bajo coste temporal

Búsqueda Metaheurística

Local Search

- **Eficientes** en memoria.
- **Inconvenientes:** no garantizan la óptima, incompletitud, ciclos, etc..

Local / Global

Heurística: guía de la búsqueda

Global Search

- **Completos y admisibles.**
- Pueden garantizar la solución óptima
- **Alto coste** en memoria.

METAHEURÍSTICAS

La idea básica metaheurística es mejorar la búsqueda local (*explotación*),

- modulando la aplicación de la heurística mediante componentes de decisión estocásticos y/o métodos bio-inspirados.
- incorporando simultáneamente ventajas de la búsqueda global (*exploración*), minimizando la probabilidad de quedar atrapados en óptimos locales.

- ✓ Utilizan poca memoria.
- ✓ Rápidas respuestas iniciales, que continúan optimizando (*any time*).
- ✓ Mejoran la búsqueda local, sin la complejidad espacial/temporal de una búsqueda global.
- ✓ Implementación sencilla. *Dificultad: Heurística y Ajuste.*

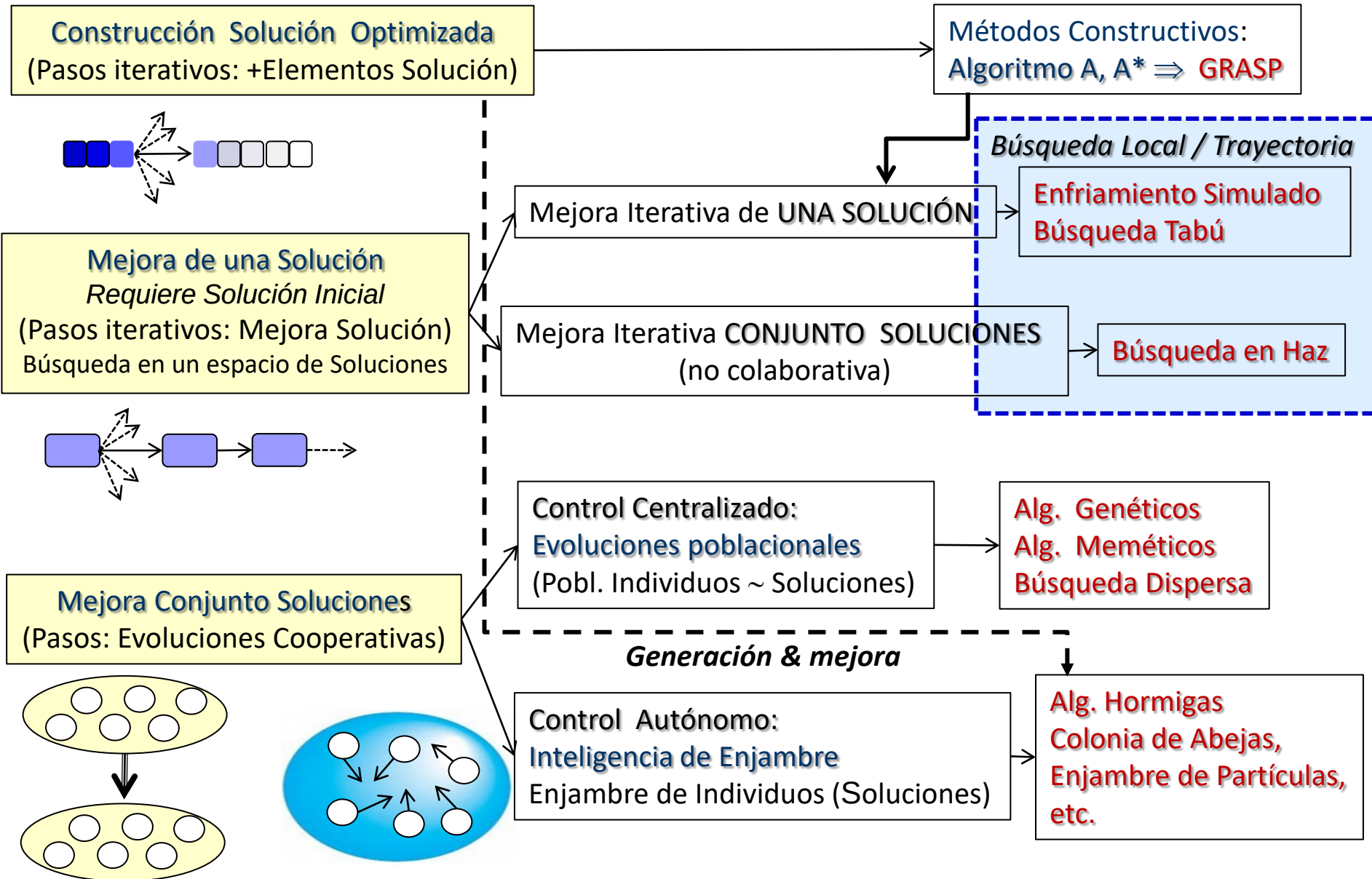
(introducidos en “*Future paths for integer programming and links to artificial intelligence*” F. Glover, 1986)

- **Métodos aproximados**, diseñados para resolver **problemas difíciles** de optimización combinatoria en tiempos razonables, los que los heurísticos clásicos no son efectivos.
- Se sitúan conceptualmente por encima de los heurísticos: Procedimientos **iterativos**, que **modifican (modulan)** la decisión de una **heurística subordinada** durante el proceso de búsqueda para producir eficientemente soluciones de alta calidad.
- Proporcionan un marco general para crear nuevos **algoritmos híbridos** combinando diferentes conceptos derivados de la **inteligencia artificial**, **comportamientos sociales** la **evolución biológica** y los mecanismos **estadísticos**: **Evolución Continua**
- **Evaluación**: Eficiencia (tiempo computacional), Eficacia (optimalidad) y Efectividad (para diversos problemas/escenarios).
 - + Obtienen soluciones muy optimizadas en tiempos razonables a problemas muy complejos, Conceptos subyacentes sencillos, implementación simple y fácilmente paralelizable, Procesos iterativos y any-time.
 - Procesos no determinísticos y aproximados, Complejo ajuste de parámetros.

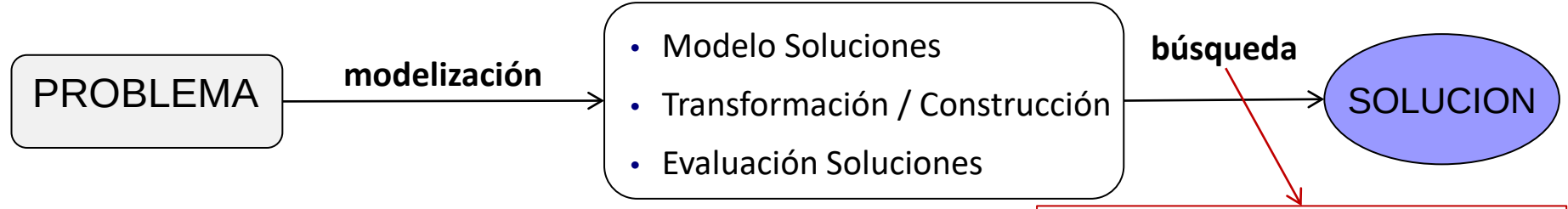
Los orígenes:

- **Enfriamiento simulado:** Kirkpatrick, S.; Gelatt Jr., C.D.; Vecchi, M.P. (1983). "Optimization by Simulated Annealing". Science 220 (4598):671–680. DOI 10.1126/science.220.4598.671
- **Búsqueda tabú:** Fred Glover and C. McMillan (1986): "The general employee scheduling problem: an integration of MS and AI". Computers and Operations Research, 1986.
- **Búsqueda por Haz Local (Beam Search):** Reddy, D. Raj, (1977): "Speech Understanding Systems:A Summary of Results of the Five-Year Research Effort. Dept. Comp. Science", Carnegie Mellon Univ. 1977.
- **Algoritmos genéticos:** John H. Holland (1975): "Adaptation in Natural and Artificial Systems". University of Michigan Press, 1975
- **Algoritmos Mémeticos:** Moscato P., (1989) "On Evolution, Search, Optimization, Genetic Algorithms and Martial Arts: Towards Memetic Algorithms". Caltech Concurrent Computation Program (report 826).
- **Scatter search:** Fred Glover (1977): "Heuristics for Integer programming Using Surrogate Constraints". Decision Sciences 8 (1): 156–166, 1977.
- **GRASP:** T.A. Feo and M.G.C. Resende (1989): "A probabilistic heuristic for a computationally difficult set covering problem". Operations Research Letters, 8:67–71, 1989.
- **Inteligencia de Enjambre:** Beni, G., Wang, J. (1989): Swarm Intelligence in Cellular Robotic Systems, Proceed. NATO Advanced Workshop on Robots and Biological Systems, Tuscany, Italy, June 26–30 (1989)
- **Colonias de hormigas:** Marco Dorigo (1992): "Optimization, Learning and Natural Algorithms" PhD thesis, Politecnico di Milano, 1992.
- **Colonia de Abejas:** Karaboga, D, (2005). An idea based on honey bee swarm for numerical optimization. Technical Report TR06, Erciyes University, Engineering Faculty, Computer Engineering Department, 2005.
- **Enjambre de partículas:** J. Kennedy & R. Eberhart (1995): "Particle Swarm Optimization". Proceedings of IEEE International Conference on Neural Networks, 1995.

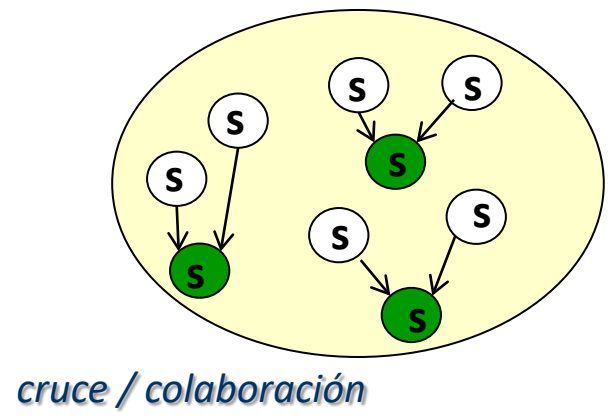
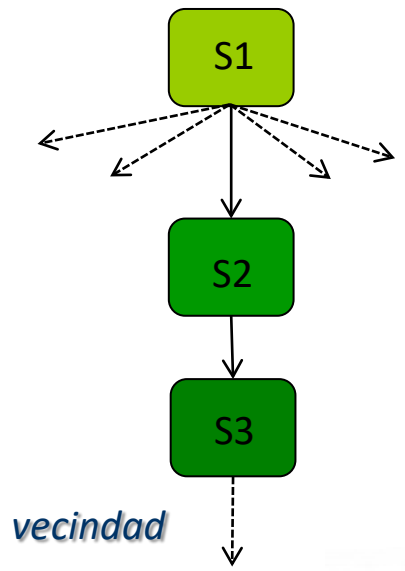
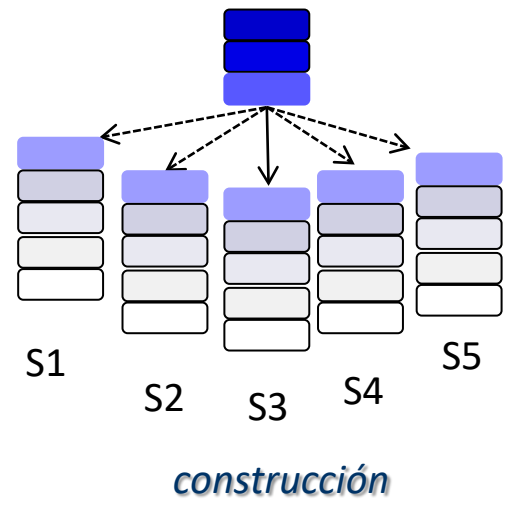
Métodos Metaheurísticos. Tipos Básicos.



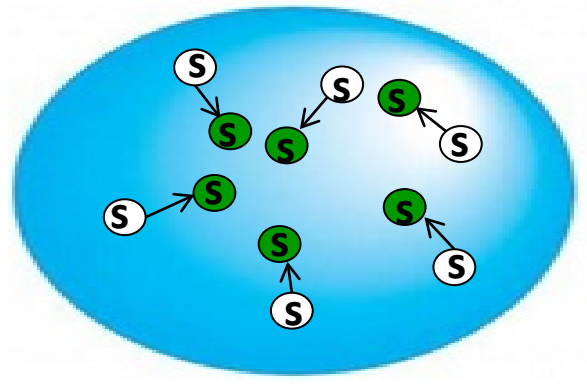
Resolución de Problemas



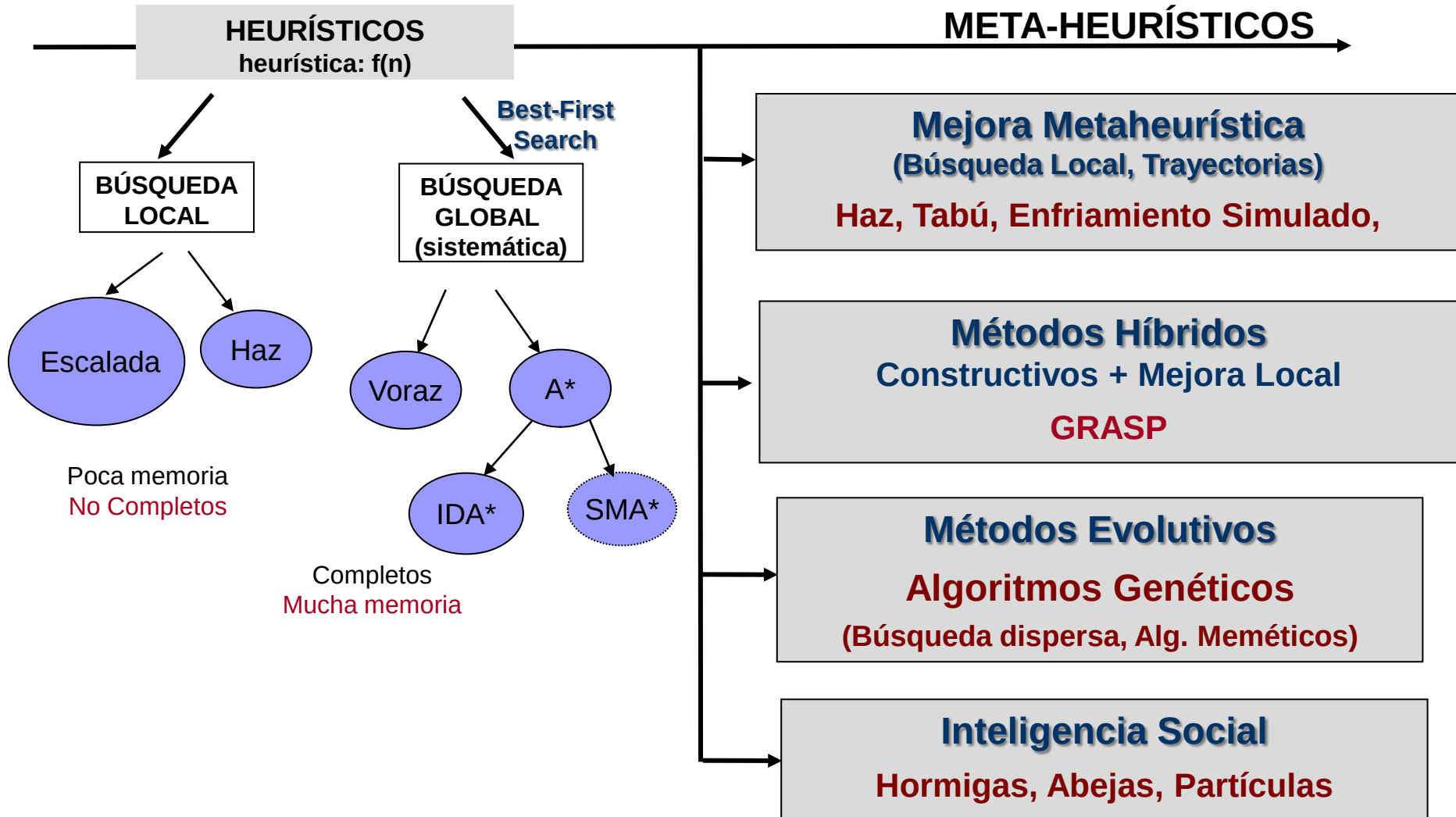
Qué padres, vecinos, sucesor,?



colaboración social



Métodos Metaheurísticos



No hay una única clasificación....

Metaheurísticas de Trayectorias:

- Búsqueda tabú
- Búsqueda local guiada
- Enfriamiento simulado, etc..

Metaheurísticas Poblacionales:

- Búsqueda dispersa
- Algoritmos evolutivos, genéticos
- Algoritmos meméticos
- Sistemas de hormigas
- Inteligencia de enjambre, etc.

Multi-arranque / Constructivos

- GRASP

Inspiradas o no en la Naturaleza

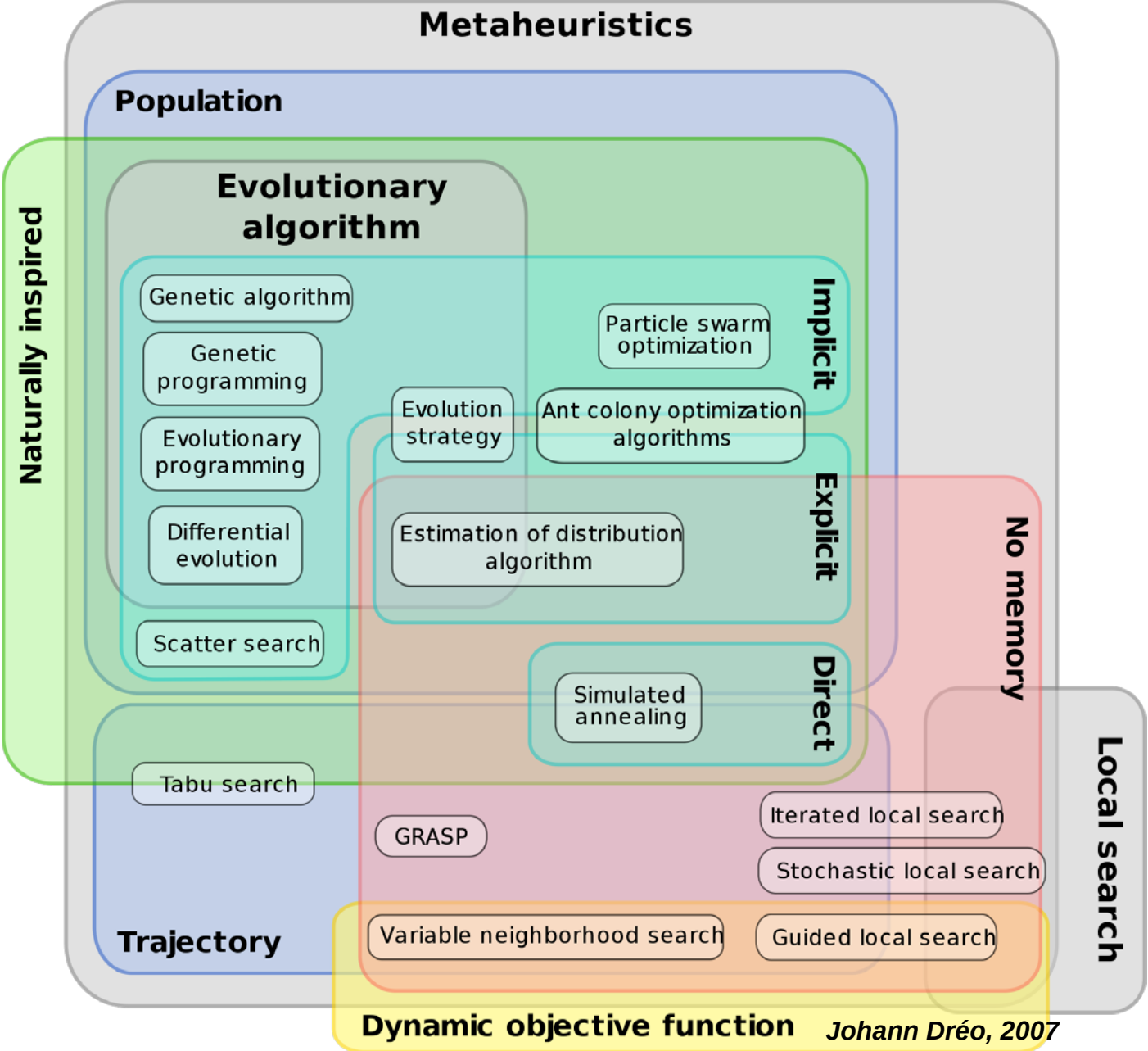
- Inspiradas: Genéticos Hormigas, Enfr. Simulado, etc.
- No Inspiradas: Búsqueda dispersa, GRASP...

Aleatorias vs. Determinísticas

- Aleatorias: Genéticos, ES, etc.
- Deterministas: Tabú, Búsqueda Dispersa, etc.

Incluso, las redes neuronales, la programación por restricciones, etc., son a veces incluidas en el término 'metaheurística'

No hay una única clasificación....



Exploración y Explotación de Soluciones

En el proceso de búsqueda, se debe hacer continuamente una elección fundamental: *exploración vs explotación*

Exploración de alternativas en el espacio de soluciones.

- La exploración busca alternativas un amplio espacio de estados $\approx$ Búsqueda global.
- Evita convergencia prematura.

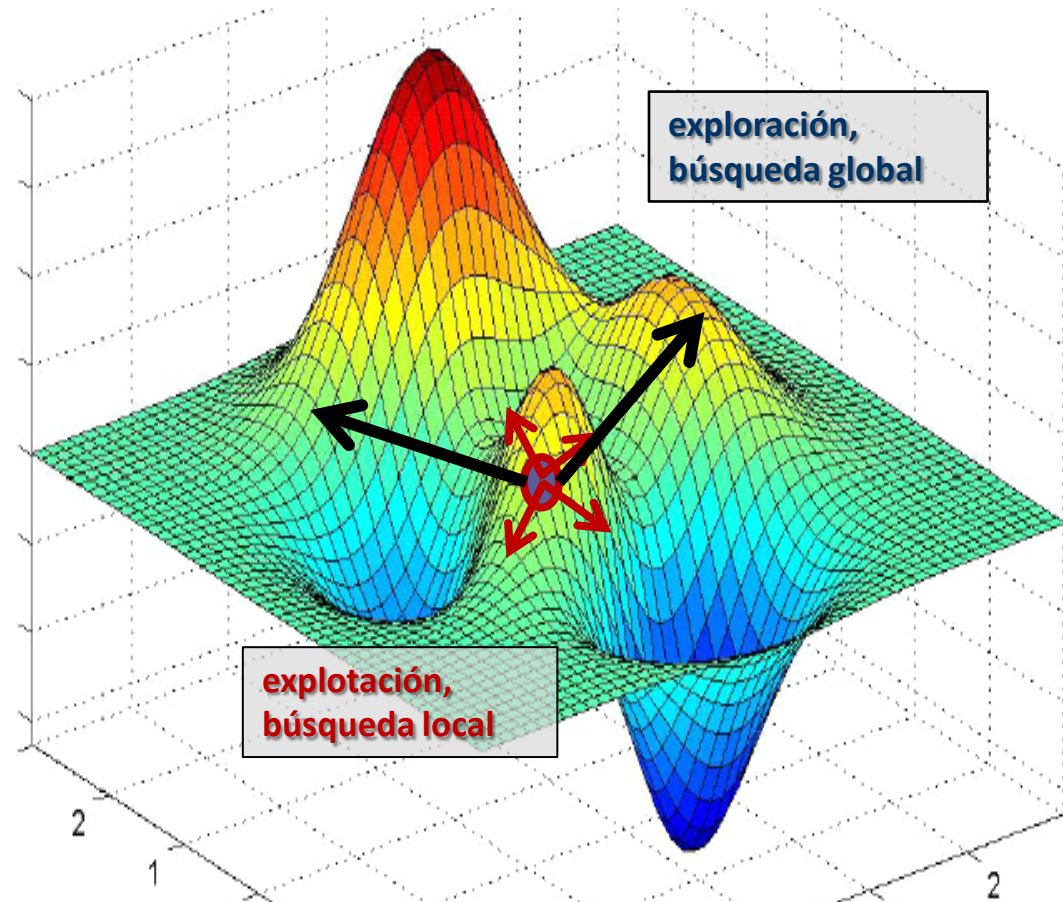
Tiende a obtener más información.

Explotación de alternativas vecinas de una buena solución

- Permite focalizar sobre el entorno de buenas soluciones (alto fitness) $\approx$ Búsqueda local en la vecindad.

Tiende a hacer la mejor decisión, dada la información actual.

Las metaheurísticas tienden a combinar y/o alternar, con mayor o menor éxito, ambas opciones (*explotación vs exploración*)



Ejemplo típico (*Falkenauer*):

Tenemos dos tragaperras. Una de las dos da premios con mayor frecuencia que la otra: Por ejemplo una da premio un 60% de las veces, la otra un 40%. Pero no se sabe cual es cada una. Lo ideal es jugar solo a la máquina buena, pero ¿cómo saberlo?

La cuestión es:

¿cómo se debe jugar para maximizar las ganancias?

Estrategia posible:

- 1) Jugar n veces la izquierda y n veces la derecha. *Exploración*
- 2) Comparar resultados y determinar mejor palanca.
- 3) Jugar solo a la máquina buena *Explotación*

Pero ¿Cuántas ($n+n$) veces probar al principio?

¿Cabe volver a probar ambas máquinas?



- La división entre las etapas de exploración/explotación es artificiosa.
- Parece mejor integrar ambas etapas, y seguir jugando ambas palancas (aunque cada vez se juegue más veces la palanca ganadora). ¿Cómo distribuir n_1 , n_2 ?
- Por ejemplo (Holland – Alg. Genéticos), propone la estrategia de que el nº de ensayos con la palanca ganadora (o probabilidad de jugarla) crezca exponencialmente respecto a la mala.

¿Y si tenemos k máquinas tragaperras?

Entornos de Desarrollo de Aplicaciones Metaheurísticas

Aunque existen, no es habitual utilizar herramientas o entornos de desarrollo.

- Procesos no complejos (iterativos) y muy dependientes del problema.

Opciones (típicamente para evaluación):

- Toolbox de Optimización en Matlab (búsqueda global, multiarranque, algoritmos genéticos, enfriamiento simulado, etc.). Global Optimization Toolbox (*entre otras librerías*)

<http://www.mathworks.es/es/products/global-optimization/index.html>

- Opt4J (Java, Libre, <http://opt4j.sourceforge.net/>).
- HeuristicLab (<http://dev.heuristiclab.com/>),
- PARADISEO (<http://paradiseo.gforge.inria.fr/>), j
- Metal (<http://jmetal.github.io/jMetal/>), etc.

Librerías de programación

- PISA (<http://www.tik.ee.ethz.ch/sop/pisa/>),
- ECJ 23 (Java, <http://cs.gmu.edu/~eclab/projects/ecj/>)
- ECF (C++ <http://ecf.zemris.fer.hr/>),
- DEAP (Python <http://deap.readthedocs.org/>), /, etc.,...

Table 1
Summary of the analysis: techniques and their properties.

Method	Modeling	Anytime	Time	Memory	Tuning	Tool
Chronological backtracking	Coded	Partial	High	Low	No	No
Branch and bound	Coded	Partial	High	Low	No	No
Mixed integer programming	Declarative	No/yes	Low	High	No	GLPK
Constraint logic programming	Declarative	No	High	Low	No	GProlog
Forward checking	Declarative	No	Medium	Medium	No	ConFlex
Synchronous backtracking	Declarative	No/yes	Medium	Medium	No	No
Asynchronous backtracking	Declarative	No	High	High	No	ADOPT
Tabu	Coded	Yes	Low	Medium	Yes	No
GRASP	Coded	Yes	Low	Medium	Yes	No
Combinatorial auctions	Coded	Partial	Medium	High	No	GLPK
Genetic algorithms	Coded	Yes	Low	Low	Yes	No
Ant colony optimization	Coded	No	Low	Medium	Yes	No

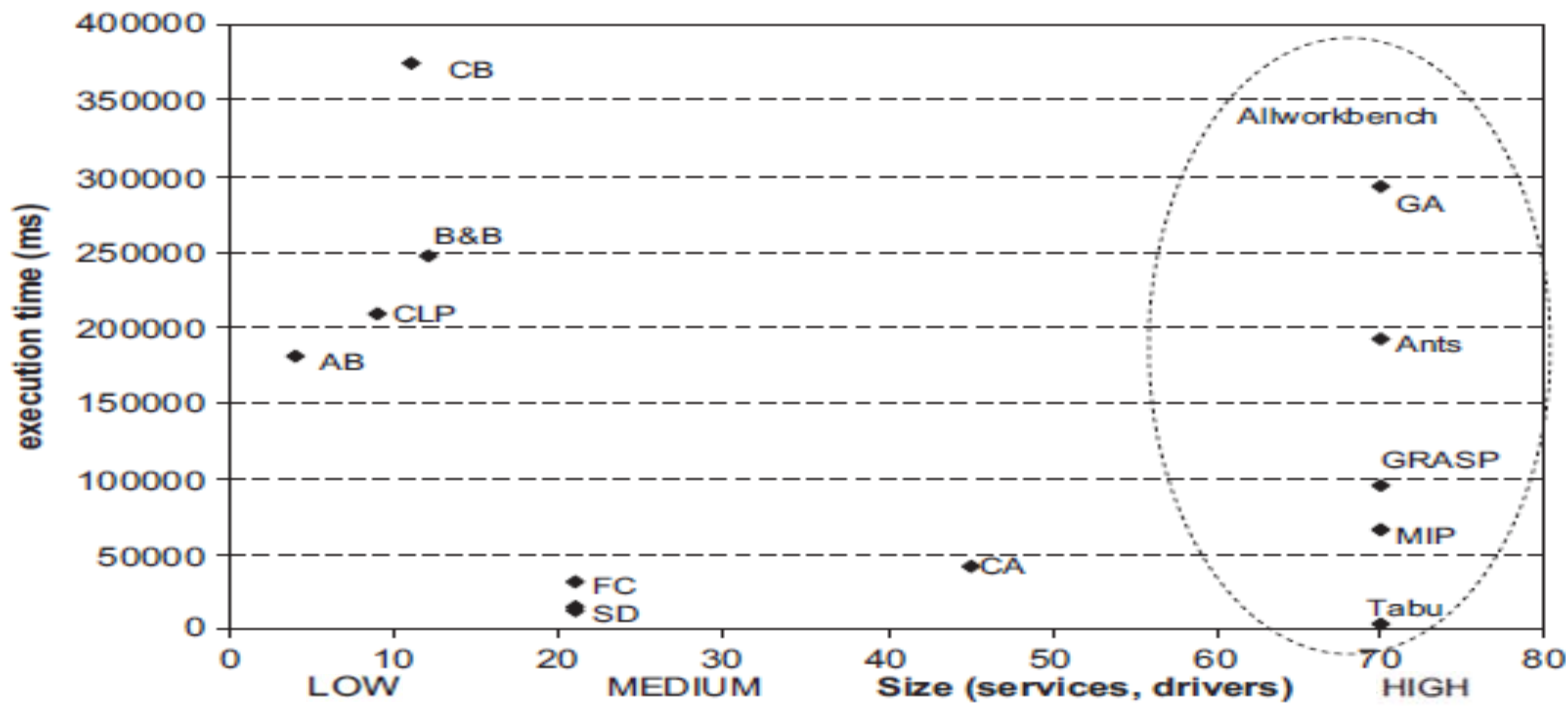


Fig. 7. Maximum problems managed by the methods and the associated execution time.

Instance	GA		ACO		SA	
	Best Distance	Time (s)	Best Distance	Time (s)	Best Distance	Time (s)
bier127	419224	~3	124651.524	~27	265289	~0.3
berlin52	11551	~1	7721.432	~4	10586	~0.2
ali535	9466	~5	1510.637	~62	6471	~0.3

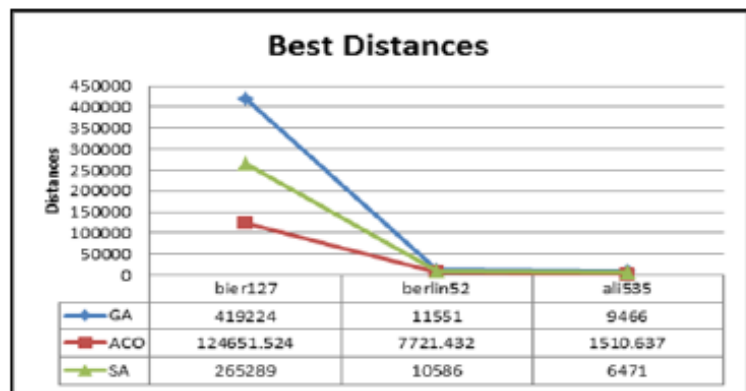


Figure 12. Evolution of GA, ACO and SA results over best distance for bier127, berlin52, and ali535

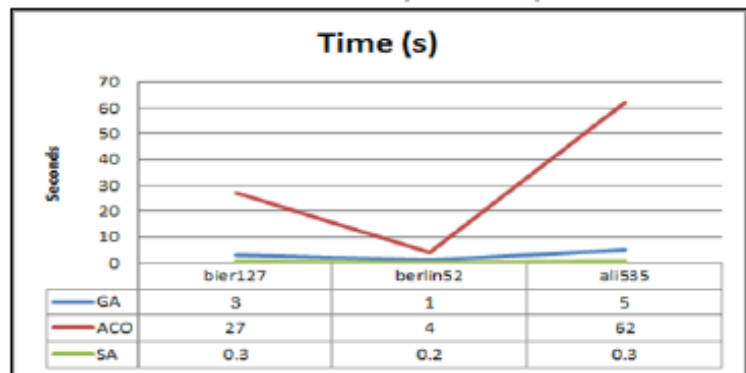


Figure 13. Evolution of GA, ACO and SA results over time(s) for bier127, berlin52 and ali535

Contents lists available at SciVerse ScienceDirect

Information Sciences

journal homepage: www.elsevier.com/locate/ins

A survey on optimization metaheuristics

Ilhem Boussaïd^a, Julien Lepagnot^b, Patrick Siarry^{b,*}

^a Université des sciences et de la technologie Houari Boumediène, Electrical Engineering and Computer Science Department, El-Alia BP 32 Bab-Ezzouar, 16111 Algiers, Algeria

^b Université Paris Est Créteil, LISSI, 61 avenue du Général de Gaulle 94010 Créteil, France

Artif Intell Rev (2015) 44:1–21
DOI 10.1007/s10462-013-9399-6

Review of state of the art for metaheuristic techniques in Academic Scheduling Problems

Chong Keat Teoh · Antoni Wibowo · Mohd Salihin Ngadiman

Distintos algoritmos/heurísticas de búsqueda/optimización

Algoritmo de las Hormigas

Enfriamiento Simulado

BÚSQUEDA DISPERSA

Búsqueda Tabú

Algoritmo A*

Algoritmo de las Abejas

Algoritmos Genéticos

GRASP

Algoritmos Meméticos

Heurística $h1(n)$

Heurística $h2(n)$

Heurística $h3(n)$

Cual es mejor?

'No free lunch' Theorem

(No free lunch theorems for optimization, Wolpert, D.H.; Macready, W.G.;
IEEE Transactions on Evolutionary Computation, 1, 1997)



***“The computational cost of finding a solution,
averaged over all problems in the class, is the same for any solution method.”***

***“For any algorithm, any elevated performance over one class of problems
is exactly paid for in performance over another class.”***

- “Todos los algoritmos (de búsqueda o de optimización) tienen la misma evaluación cuando se promedia su aplicación sobre el conjunto de problemas”.

“Para cada algoritmo de búsqueda/optimización, cualquier superioridad que pudiera tener sobre una clase de problemas es exactamente penalizada sobre las otras clases de problemas”.

- “En un método de búsqueda, todas las funciones heurísticas (de búsqueda u optimización) tienen la misma evaluación cuando se promedian sus aplicaciones sobre el conjunto de funciones de coste”.

“La superioridad de una heurística, definida sobre una función de coste, es exactamente penalizada cuando se aplica sobre las otras funciones de coste”.

Por lo tanto, **para cualquier algoritmo de búsqueda/optimización, su eventual alto rendimiento sobre una clase concreta de problemas, es exactamente penalizado sobre otra clase de problemas.**

Diversas discusiones relacionadas: <http://www.no-free-lunch.org/>

Metáfora del Teorema “No Free Lunch”

Cada “restaurante” (método/heurística) tiene una carta, que asocia cada “plato” (problema/función de coste) con su “precio” (evaluación del método/heurística en la resolución del problema).

Las cartas de los restaurantes tienen los mismos platos y solo difieren en el precio de los mismos.

Restaurante (Método/Heurística)

Plato_i (Problema_i) → Precio_i (Evaluación_i)

Restaurante: Método/Heurística

Plato: Problema / Función de Coste

Precio: Evaluación de la Heurística al problema

¿Qué restaurante (método) elegir para mi plato preferido (problema) para pagar menos (criterio)?



Restaur: Algoritmos Genéticos	
Carne.1	8,4
Cereales.2	3.2
Pescado.3	7.3
Restaur: Enfriamiento Simulado	
Carne.1	6.8
Cereales.2	4.5
Pescado.3	8.2
Restaur: Búsqueda Tabú	
Carne.1	4.1
Cereales.2	6.4
Pescado.3	9.2

- Para un **carnívoro** (un tipo de problema), habrá posiblemente un restaurante (método/heurística) más económico (mejor) que otro.

PERO:

- Pero si un **vegetariano** (otro problema), come siempre con un carnívoro que elige su restaurante más económico, tendrá probablemente que pagar más por su comida.

Es decir, para elegir el mejor restaurante, deben conocerse los gustos de la persona (**problema/función de coste**) y qué cuesta cada tipo de comida en uno u otro restaurante (**restaurante/heurística**) .

Para una adecuada resolución de un problema, se requiere usar información del problema/función de coste (plato) para seleccionar el mejor método/heurística (restaurante).

Restaurante: Método/Heurística
Plato: Problema / Función de Coste
Precio: Evaluación

Carta: Algoritmos Genéticos

Carne.1 8,4

Cereales.2 3.2

Pescado.3 7.3



Carta: Enfriamiento Simulado

Carne.1 6.8

Cereales.2 4.5

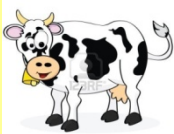
Pescado.3 8.2

Carta: Búsqueda Tabú

Carne.1 4.1

Cereales.2 6.4

Pescado.3 9.2



Distintos algoritmos/heurísticas de búsqueda/optimización

Algoritmo de las Hormigas
Enfriamiento Simulado

Algoritmos Genéticos
GRASP

Búsqueda Tabú

Algoritmo A*

Algoritmo de las Abejas

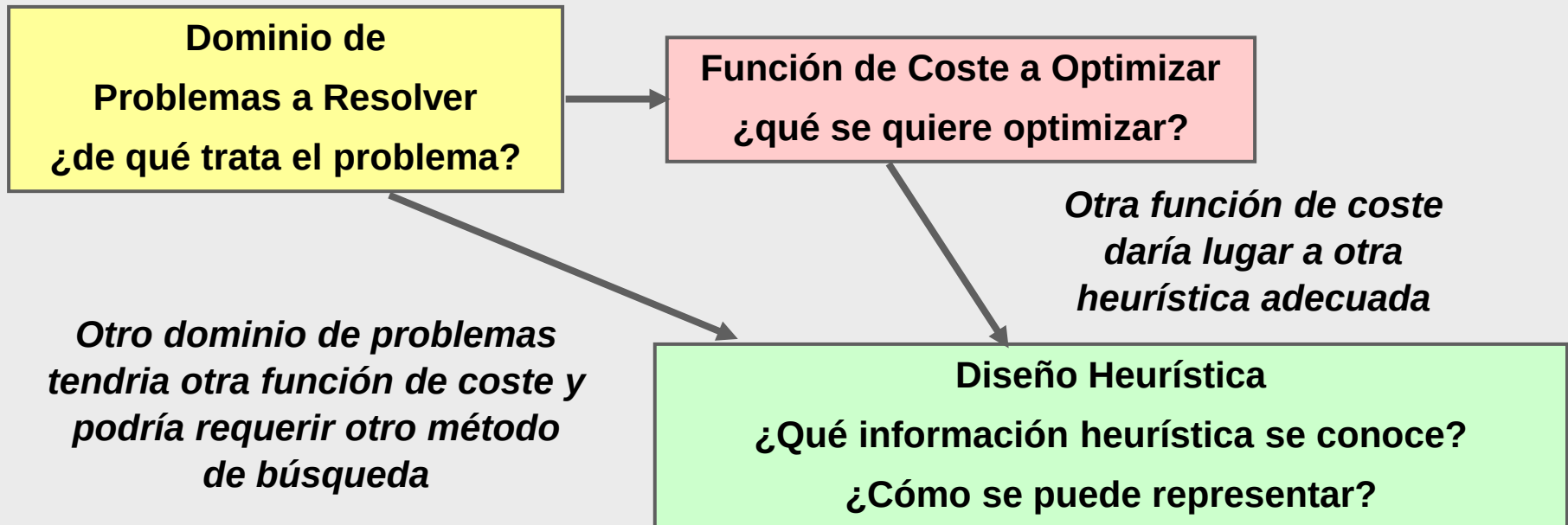
Heurística $h1(n)$

Heurística $h2(n)$

Heurística $h3(n)$

Cual es mejor?

- No se sabe a priori.
- Se necesita información del dominio de problemas a resolver y de lo que se requiere buscar/optimizar



En conclusión,

- No hay un método/heurística adecuado para todos los problemas, o para todas las funciones de coste aplicables a un problema.
- La bondad de un método de búsqueda / heurística está determinada por su capacidad de captar el conocimiento relevante en el dominio para la mejor resolución del problema concreto.

Incluso, más allá.... (D. H. Wolpert, NASA-ARC-05-097)

LIMITS ON MATH, SCIENCE AND BEYOND

- 1) *NFL for supervised learning formalizes Hume:
Science cannot give guarantees about future experiments
based on results of previous experiments.*
- 2) *Godel's theorems say math cannot give guarantees
about its own conclusions.*
- 3) *No matter what simulation program it runs, no computer can
give guarantees about any future physical experiment.*

*More generally, no system - even the universe itself - can give
guarantees about prediction, control or observation.*

IMPOSSIBILITY OF INFERENCE

- *No device can infer itself.*
- *No two distinguishable devices can infer each other*

- 1) *The universe may contain one device that can predict the rest of
the universe - but no more than one.*
- 2) *If you have many distinguishable devices, at most one can infer
all the others: a God device.
I.e., at most one device that can (infallibly) observe / predict /
control all distinguishable others: "Monotheism".*
- 3) *A time-translated copy of a God device cannot be a God device.
I.e., God can only be infallible once: "Intelligent design".*

Tema-1: Búsqueda de soluciones en IA. Heurísticos y Metaheurísticos.

El problema de la búsqueda de soluciones.

Satisfabilidad y Optimalidad. Explosión combinatoria.

Tipología de la Búsqueda:

Búsqueda Global (sistemática), Búsqueda Local.

Métodos Constructivos, Métodos de Mejora, Métodos Poblacionales.

Búsqueda Heurística.

Tipos. Algoritmos A, A*. Variantes.

Metaheurísticos.

Tipología Metaheurísticas. Exploración vs Explotación.

Elección de una metaheurística. No free lunch.